

# AIRBNB-HOST PREDICTION

## - FINAL REPORT

### **TEAM**

Abhishek J,  
Abhishek B,  
Aditya Bose,  
Naganaik NS,  
SoundhariyaV

**Table of Contents:**

|                                                |    |
|------------------------------------------------|----|
| Industry Review                                | 2  |
| Current Practices                              | 3  |
| Background Research                            | 4  |
| Literature Survey - Publications, Applications | 4  |
| Data Dictionary                                | 4  |
| Variable Categorization Summary                | 6  |
| Data Attribute Details                         | 6  |
| Project Justification                          | 7  |
| Data Exploration                               | 7  |
| Relationship between the variables             | 8  |
| Univariate analysis                            | 8  |
| Bi-variate analysis                            | 12 |
| Outliers                                       | 15 |
| Class imbalance                                | 17 |
| Correlation                                    | 17 |
| Statistical Tests                              | 19 |
| Model Deployment                               | 20 |
| Logistic Regression                            | 20 |
| Decision Tree Classifier                       | 22 |
| Decision Tree Classifier-Tuned                 | 25 |
| Random Forest Classifier                       | 26 |
| Random Forest Classifier -Tuned                | 27 |
| Ada Boost Classifier                           | 29 |
| Ada Boost Classifier-Tuned                     | 30 |
| Gradient Boosting Classifier                   | 32 |
| Gradient Boosting Classifier-Tuned             | 33 |
| XG Boost Classifier                            | 35 |
| XG Boost Classifier -Tuned                     | 37 |
| KNN Classifier                                 | 39 |
| Model Summary                                  | 40 |
| Best Model Selection                           | 42 |
| Recommendations to the Business                | 43 |
| Host Experience Prediction (using Streamlit)   | 45 |
| Future Scope                                   | 47 |

## Industry Review – Current practices, Background Research

Airbnb is one of the world's largest peer-to-peer accommodation platforms, enabling individual hosts to list properties and travelers to book short-term stays globally. The platform operates at scale across diverse geographies, property types, and host experience levels. Since hosts directly influence listing quality, pricing, availability, and guest satisfaction, understanding host behavior plays a critical role in sustaining marketplace trust and long-term platform growth.

Most platforms provide onboarding materials such as help centers, email guides, and general hosting tips. While these resources are useful, they are largely passive and not personalized to individual host needs. Existing onboarding campaigns typically target all hosts equally, resulting in inefficient resource allocation and limited impact on hosts who genuinely require additional guidance or intervention.

Airbnb automatically classifies hosts based on their activity and history.

There is no manual tagging — it's data-driven.

Typical signals used:

- How recently the host joined
- Number of completed stays
- Number of reviews and ratings
- Response rate and response time
- Profile completeness and verification

Hosts with little or no history are treated as **new hosts**; consistent performers are treated as **experienced hosts**.

### Background Research & ML Evidence

Prior research shows that early listing performance indicators—such as review ratings, response time, pricing accuracy, and listing completeness—are strong predictors of long-term success. Machine learning models are already widely used for pricing optimization, demand forecasting, and ranking listings. This establishes strong empirical evidence that similar structured features can effectively classify host experience levels

### Useful Data for Detecting New Hosts

Key signals include host profile attributes such as account age, number of active listings, response behavior, and verification status. Listing-level features such as price deviation from local averages, review availability, and booking restrictions also provide valuable insights. Predicting host experience enables platforms to detect risk, personalize onboarding, optimize search rankings, and improve guest experience across cities.

## Literature Survey - Publications, Applications

Predicting whether an Airbnb host is new or experienced falls under the broader research domains of:

#### **Trust & Reputation Systems**

Several academic studies have examined host-related characteristics to understand performance, trust, and booking outcomes on peer-to-peer accommodation platforms such as Airbnb. These studies provide strong empirical support for predicting host experience using structured platform data.

#### **Zhang et al. (2020) – “Understanding Airbnb Host Behavior”**

Zhang et al. analyzed host tenure, responsiveness, and listing activity to study host reliability and long-term performance. Their findings show that host tenure—defined as the duration since a host joined the platform—is a strong predictor of guest satisfaction and booking stability. Experienced hosts demonstrated faster response times, higher consistency in service quality, and fewer cancellations. This study directly supports the inclusion of temporal variables such as `host_since` when distinguishing new and experienced hosts.

#### **Li et al. (2019) – “Host Experience and Booking Likelihood”**

Li et al. examined how host experience impacts booking probability using machine learning models. The study found that new hosts receive significantly fewer bookings compared to experienced hosts due to lack of reviews, incomplete profiles, and pricing inefficiencies. The authors demonstrated that supervised learning models can effectively classify hosts into new and experienced categories using platform metadata, validating the feasibility of the classification task addressed in this project.

#### **Ert, Fleischer & Magen (2016) – “Trust and Reputation in Airbnb”**

This study focused on trust formation in peer-to-peer marketplaces. The authors identified that host profile features such as profile photographs, response rates, verification status, and self-descriptions strongly influence guest trust—especially for new hosts without reviews. Their work highlights the importance of non-price signals in compensating for lack of experience, reinforcing the predictive value of host profile attributes for identifying inexperienced hosts.

#### **Relevance to This Project**

Together, these studies establish that host tenure, responsiveness, and profile completeness are critical determinants of host performance and guest trust. They provide strong theoretical and empirical justification for

## greatlearning

building machine learning models to predict whether a host is new or experienced using Airbnb listing and host-level features.

These rely heavily on:

host\_since

host\_response\_rate

host\_acceptance\_rate

host\_identity\_verified

All these features are used to judge whether a host is reliable or *new*.

### **Past & Ongoing Research**

#### **Host Tenure as a Predictor**

Research conducted between 2017 and 2023 consistently identifies host tenure (host\_since) as one of the strongest predictors for identifying new hosts on Airbnb and similar platforms. Studies show that new hosts typically exhibit slower response times due to unfamiliarity with platform operations and guest communication norms. They also tend to accept fewer booking requests, often driven by uncertainty or lack of confidence. Additionally, new hosts generally price their listings more conservatively to attract initial bookings and reviews. Longitudinal studies indicate that inexperienced hosts are more likely to deactivate or abandon listings early, highlighting tenure as a key indicator of host stability and platform retention.

#### **Text Mining on Host Description**

Ongoing research explores the use of Natural Language Processing (NLP) techniques to extract behavioral signals from unstructured host descriptions and profile texts. Phrases such as “I’m new to Airbnb,” “first time hosting,” or “just started hosting” provide explicit indicators of host experience. Beyond experience signals, researchers also extract geographic information from free-text fields containing informal or ambiguous locations, such as “I live between Paris and Barcelona.”

Techniques such as keyword extraction, named entity recognition (NER), and text normalization are used to convert unstructured descriptions into structured features that can enhance host classification models.

### **Data Dictionary and Pre-processing Data Analysis:**

**Table: Airbnb Dataset – Column Details**

| No | Column Name               | Non-Null Count | Data Type | Description                             |
|----|---------------------------|----------------|-----------|-----------------------------------------|
| 1  | listing_id                | 279712         | int64     | Unique identifier for each listing      |
| 2  | name                      | 279537         | object    | Name/title of the Airbnb listing        |
| 3  | host_id                   | 279712         | int64     | Unique identifier of the host           |
| 4  | host_since                | 279547         | object    | Date when the host joined Airbnb        |
| 5  | host_location             | 278872         | object    | Location provided by the host           |
| 6  | host_response_time        | 150930         | object    | Typical response time of the host       |
| 7  | host_response_rate        | 150930         | float64   | Percentage of messages responded to     |
| 8  | host_acceptance_rate      | 166625         | float64   | Percentage of booking requests accepted |
| 9  | host_is_superhost         | 279547         | object    | Indicates if the host is a Superhost    |
| 10 | host_total_listings_count | 279547         | float64   | Total listings owned by the host        |
| 11 | host_has_profile_pic      | 279547         | object    | Whether host has a profile picture      |
| 12 | host_identity_verified    | 279547         | object    | Whether host identity is verified       |
| 13 | neighbourhood             | 279712         | object    | Neighbourhood of the listing            |
| 14 | district                  | 37012          | object    | District of the listing                 |
| 15 | city                      | 279712         | object    | City where the listing is located       |
| 16 | latitude                  | 279712         | float64   | Latitude coordinate                     |
| 17 | longitude                 | 279712         | float64   | Longitude coordinate                    |
| 18 | property_type             | 279712         | object    | Type of property                        |
| 19 | room_type                 | 279712         | object    | Type of room offered                    |
| 20 | accommodates              | 279712         | int64     | Number of guests accommodated           |
| 21 | bedrooms                  | 250277         | float64   | Number of bedrooms                      |
| 22 | amenities                 | 279712         | object    | Amenities provided                      |
| 23 | price                     | 279712         | int64     | Price per night                         |

## greatlearning

| No | Column Name                 | Non-Null Count | Data Type | Description                             |
|----|-----------------------------|----------------|-----------|-----------------------------------------|
| 24 | minimum_nights              | 279712         | int64     | Minimum nights required                 |
| 25 | maximum_nights              | 279712         | int64     | Maximum nights allowed                  |
| 26 | review_scores_rating        | 188307         | float64   | Overall review score                    |
| 27 | review_scores_accuracy      | 187999         | float64   | Accuracy rating                         |
| 28 | review_scores_cleanliness   | 188047         | float64   | Cleanliness rating                      |
| 29 | review_scores_checkin       | 187941         | float64   | Check-in experience rating              |
| 30 | review_scores_communication | 188025         | float64   | Host communication rating               |
| 31 | review_scores_location      | 187937         | float64   | Location rating                         |
| 32 | review_scores_value         | 187927         | float64   | Value-for-money rating                  |
| 33 | instant_bookable            | 279712         | object    | Indicates if instant booking is enabled |

### Variable Categorization Summary

Count of Numeric and Categorical Variables

| Variable Type          | Count     | Description                                                                                           |
|------------------------|-----------|-------------------------------------------------------------------------------------------------------|
| Numeric Variables      | 19        | Includes integer and continuous variables such as prices, counts, coordinates, and review scores      |
| Categorical Variables  | 14        | Includes text and binary variables such as host attributes, location details, and room/property types |
| <b>Total Variables</b> | <b>33</b> | Total number of variables in the dataset                                                              |

### Target column

In this study, the target variable `isnewhost` is used to distinguish between new hosts and experienced hosts on the Airbnb platform. The variable is derived from the host's total number of listings.

Hosts with only one listing are classified as new hosts, while hosts with more than one listing are classified as experienced hosts. This logic is based on the assumption that managing multiple listings requires prior experience, familiarity with platform policies, and operational maturity.

The binary nature of the `isnewhost` variable makes it suitable for a classification problem, allowing machine learning models to learn patterns associated with host experience using available listing and host attributes.

This approach aligns with real-world platform practices, where historical activity and scale of operations are key



indicators used to differentiate new participants from experienced ones for targeted support, monitoring, and optimization.

### **Potential Alternate Data Sources**

Although the current analysis relies solely on the Airbnb dataset provided, additional external data sources could potentially enhance the identification of new versus experienced hosts if incorporated in future work. These data sources were not included in the present study due to availability and scope limitations.

| Potential Supplementary Data Sources |                                            |                                                                      |
|--------------------------------------|--------------------------------------------|----------------------------------------------------------------------|
| Data Source                          | Example Columns                            | Potential Value                                                      |
| Host Activity Data                   | login_frequency, calendar_update_frequency | Could indicate host engagement and operational maturity              |
| Guest Booking Behavior               | cancellation_rate, repeat_guest_ratio      | Could reflect guest trust and host reliability over time             |
| Location & Market Indicators         | average_area_price, demand_index           | Could help distinguish host performance from location-driven effects |

### **Explanation**

Incorporating such external data could provide deeper behavioral and contextual insights that are not captured in listing-level attributes alone. However, since these datasets are not publicly available or directly linked to the core dataset, they were excluded from this analysis and are suggested only as potential extensions for future research.

### **Project Justification**

#### **Project Statement:**

The objective of this project is to identify new and experienced Airbnb hosts using host and listing attributes, enabling better understanding of host behavior on the platform.

Complexity Involved:

The project involves handling a large, high-dimensional dataset with missing values, mixed variable types, and class imbalance, requiring careful preprocessing and model selection to avoid overfitting.

Project Outcome / Value:

- Commercial: Helps platforms like Airbnb provide targeted onboarding support to new hosts and improve guest experience.
- Academic: Demonstrates application of classification models and feature analysis on real-world data.
- Social: Supports improved service quality and trust in peer-to-peer accommodation platforms.

## **EXPLORATORY DATA ANALYSIS**

### **Relationship Between the Variables**

The variables in the dataset show meaningful relationships that help distinguish between new and



experienced hosts (isnewhost).

**Host-related variables** such as `host_total_listings_count`, `host_is_superhost`, `host_response_rate`, and `host_acceptance_rate` are strongly related to host experience. Experienced hosts typically manage multiple listings, respond faster to guest inquiries, and maintain higher acceptance rates, whereas new hosts often show incomplete engagement patterns.

**Review-related variables** including `review_scores_rating`, `review_scores_cleanliness`, and `review_scores_communication` tend to improve with host experience. New hosts usually have fewer or no reviews, while experienced hosts accumulate higher and more stable review scores over time.

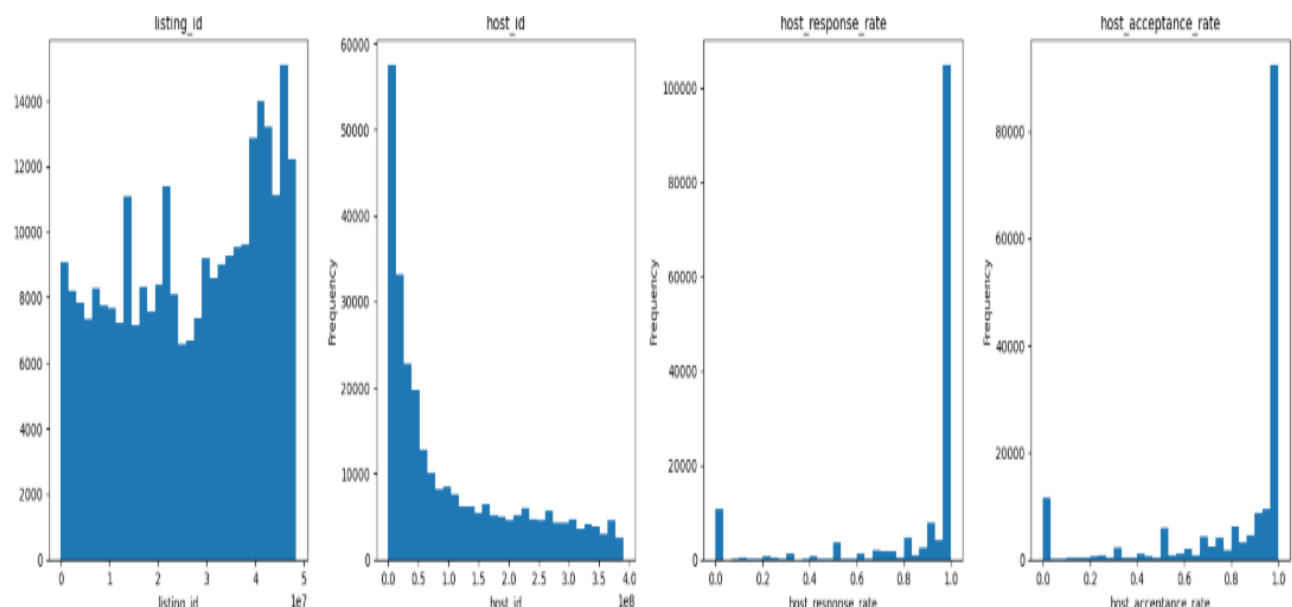
**Listing and pricing variables** such as `price`, `minimum_nights`, `accommodates`, and `bedrooms` reflect operational maturity. Experienced hosts often optimize pricing and booking rules based on demand, whereas new hosts may misprice or set restrictive booking conditions.

**Location variables** (city, neighbourhood, latitude, longitude) influence listing performance but are not direct indicators of host experience. These variables interact with pricing and demand but must be interpreted carefully to avoid location-driven bias.

**Trust and verification variables** like `host_identity_verified`, `host_has_profile_pic`, and `instant_bookable` are more frequently enabled by experienced hosts, indicating higher confidence and platform familiarity.

## Uni-variate Analysis:

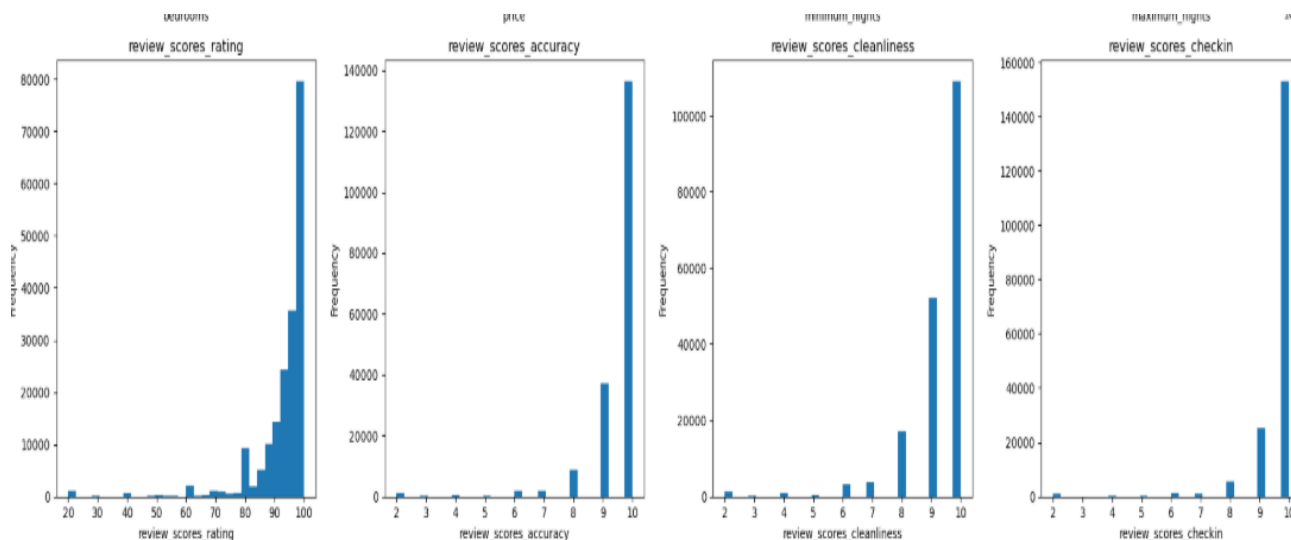
### Distribution of numeric columns



The univariate distributions reveal clear differences between identifier variables and behavioral predictors. The variables `listing_id` and `host_id` display wide and skewed numeric ranges that primarily reflect platform growth and record indexing rather than host behavior, making them unsuitable for predictive modeling. In contrast, `host_response_rate` and `host_acceptance_rate` show strong concentration near 100%,

indicating that most experienced hosts respond to and accept the majority of booking requests.

Lower values and missing entries in these variables are more common among new or inactive hosts, making them highly informative for distinguishing host experience levels. Therefore, while identifier variables should be excluded from the model, behavioral rate variables are strong predictors, with their missingness itself carrying meaningful information about host experience.



These univariate distributions show that all review-related variables are highly right-skewed and tightly concentrated at the upper end of their scales, indicating generally positive guest feedback across Airbnb listings. The `review_scores_rating` variable clusters heavily between 90 and 100, suggesting that most hosts—especially experienced ones—maintain high overall ratings. Similarly, `review_scores_accuracy`, `review_scores_cleanliness`, and `review_scores_checkin` exhibit strong spikes near values 9 and 10, reflecting consistently high performance on specific service dimensions.

## Distribution of categorical columns

### 1. Host Identity Verified

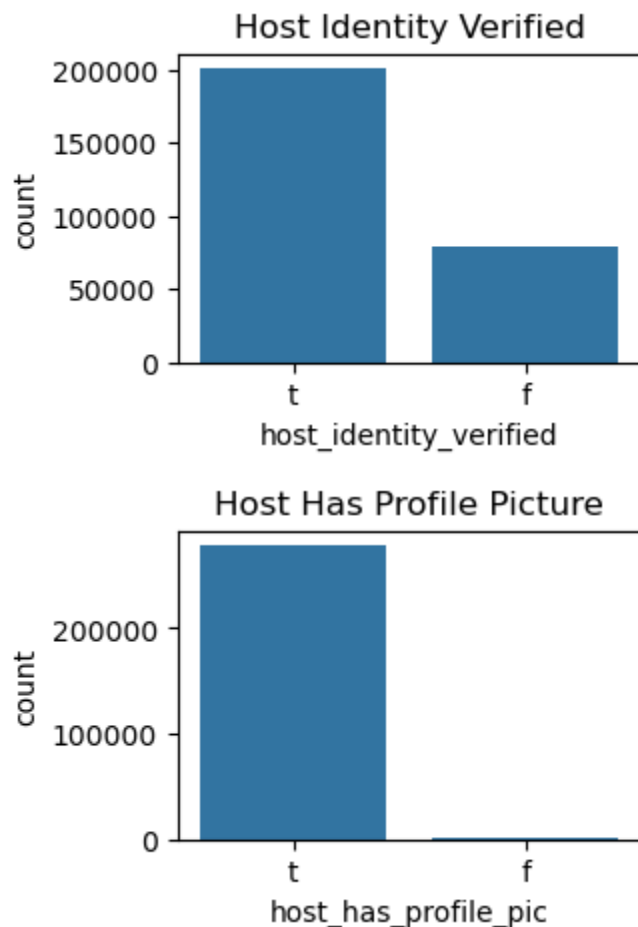
The bar chart shows that a large majority of hosts have their identity verified (t), while a smaller proportion remain unverified (f).

This indicates that identity verification is a common practice on the platform, likely encouraged by Airbnb to build guest trust. The presence of unverified hosts suggests that verification may be optional or completed later in a host's lifecycle.

### 2. Host Has Profile Picture

This plot shows an overwhelming dominance of hosts with profile pictures (t), with very few hosts lacking one (f).

This suggests that adding a profile picture is either strongly recommended or almost mandatory, and most hosts comply early.



### **1. Host Is Superhost (%)**

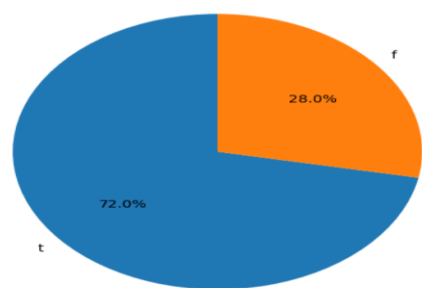
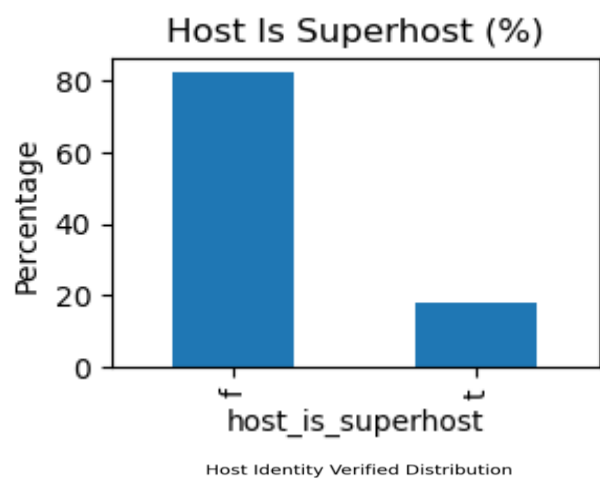
The bar chart shows that only a small proportion of hosts are Superhosts, while the majority are not. Since Superhost status is earned over time based on consistent performance, this variable strongly reflects host experience and platform maturity. New hosts are unlikely to be Superhosts, making this a strong indicator for distinguishing between new and experienced hosts.

### **2. Host Identity Verified Distribution**

The pie chart indicates that most hosts are identity verified, with a smaller portion remaining unverified. This suggests that identity verification is widely adopted across the platform, serving primarily as a trust and safety measure rather than a clear differentiator of host experience.

### **3. Distribution of Total Listings**

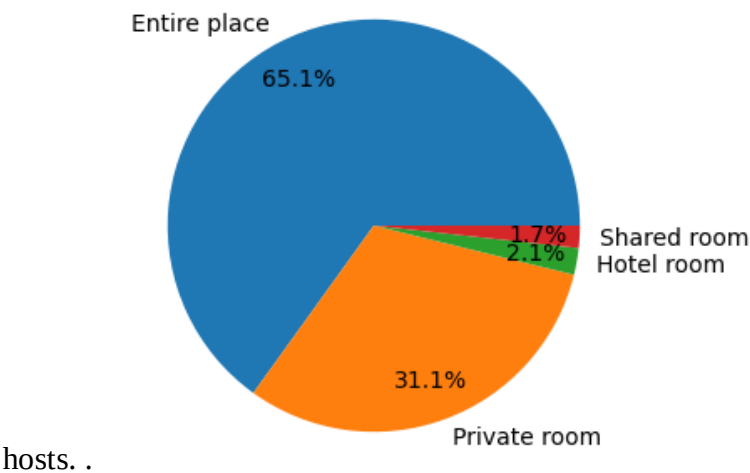
The distribution of total listings shows that most hosts manage a small number of listings, while a smaller segment controls multiple properties. Hosts with higher listing counts are typically more experienced or professional, which directly supports the derivation of the target variable `isnewhost` based on total listings.



Distribution of host identity verified Distribution

Host Identity Verified Distribution

The pie chart represents the distribution of identity verification status among hosts. A large majority of hosts are identity verified, while a smaller portion are not. This suggests that identity verification is a common trust requirement on the platform and is not, by itself, a strong differentiator between new /exp Room Type Share on Airbnb



## Room Type Distribution on Airbnb

The pie chart shows that “Entire place” listings dominate the platform, accounting for about 65% of all listings. This indicates that most hosts offer complete properties, which are generally preferred by guests seeking privacy and flexibility.

Private rooms form the second largest category at around 31%, suggesting a significant segment of hosts who rent out part of their residence, often at a lower price point.

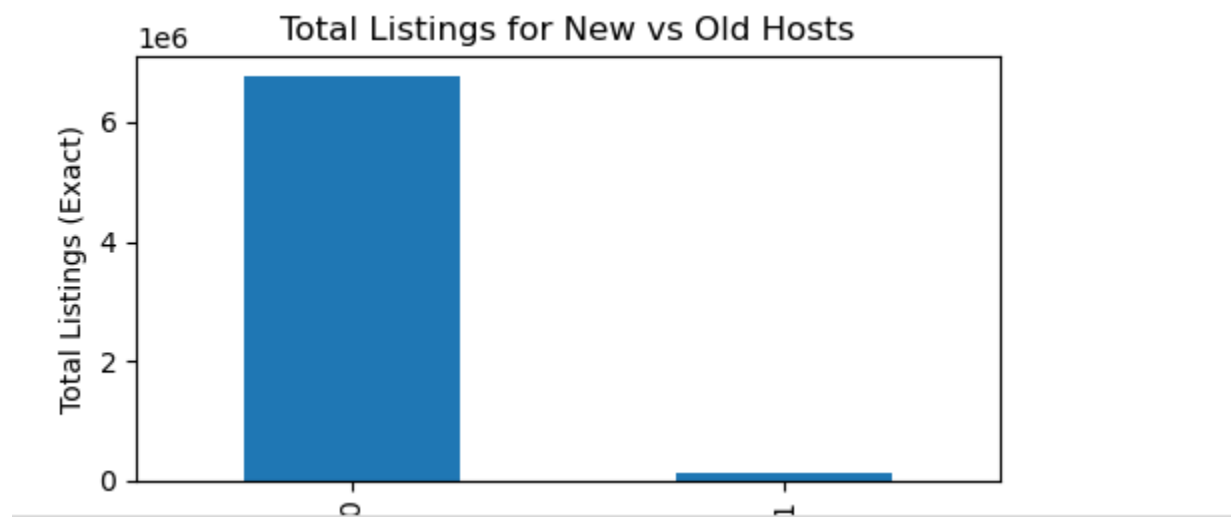
In contrast, hotel rooms (~2%) and shared rooms (~2%) represent a very small share of listings. This implies that Airbnb is primarily driven by individual hosts rather than traditional hotel-style accommodations or shared living arrangements.

## Bi-variate Analysis:

### Total Listings for New vs Old Hosts

The bar chart compares the total number of listings managed by new hosts (isnewhost = 1) and experienced hosts (isnewhost = 0). The chart shows a very large difference between the two groups, with experienced hosts contributing the overwhelming majority of total listings on the platform.

This pattern is expected because experienced hosts often manage multiple properties, while new hosts typically start with a single listing. Since the target variable isnewhost was derived using the total listings count, this strong separation validates the logic used to define the target variable.



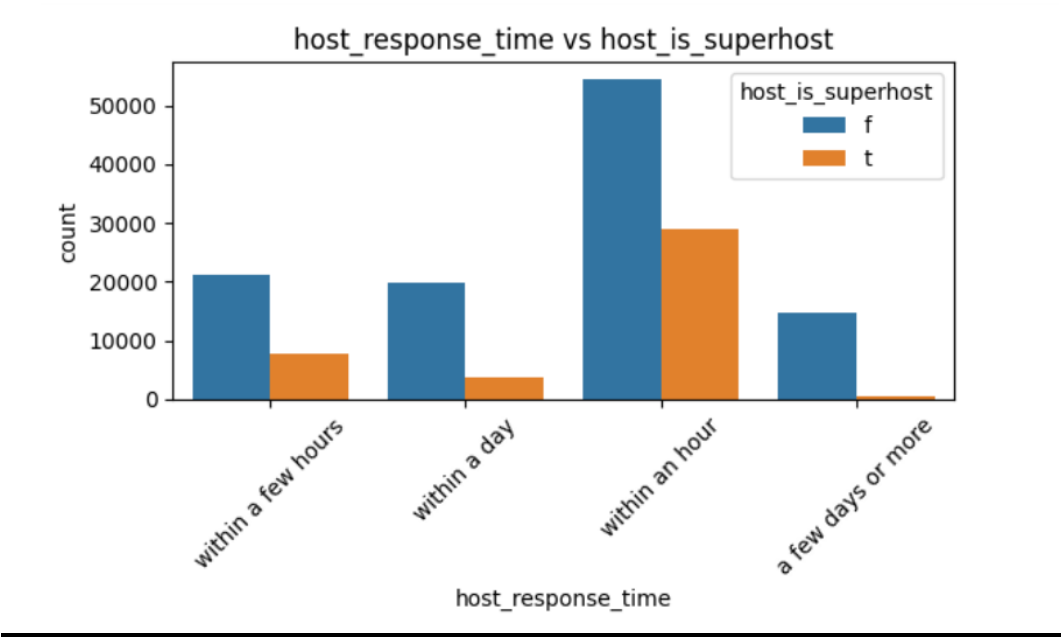
### Host Response Time vs Superhost Status

The chart compares host response time with Superhost status. It shows that hosts who respond within an hour form the largest group overall, and this category has a much higher proportion of Superhosts compared to slower response categories.

As response time increases (from *within a few hours* to *within a day* and *a few days or more*), the number

# greatlearning

of Superhosts drops sharply. Very few Superhosts fall into the slowest response category, indicating that delayed responses are uncommon among high-performing hosts.



## Host response time vs is\_new\_host

The table shows the percentage distribution of host response times for experienced hosts (isnewhost = 0) and new hosts (isnewhost = 1).

- Experienced hosts (isnewhost = 0) are more likely to respond quickly, with the highest percentages in within an hour (~70%) and within a few hours (~62%). This indicates higher engagement and operational maturity.
- New hosts (isnewhost = 1) show a higher presence in slower response categories, particularly a few days or more (~53%) and within a day (~45%). This suggests that new hosts may still be learning platform workflows or are less consistently active.

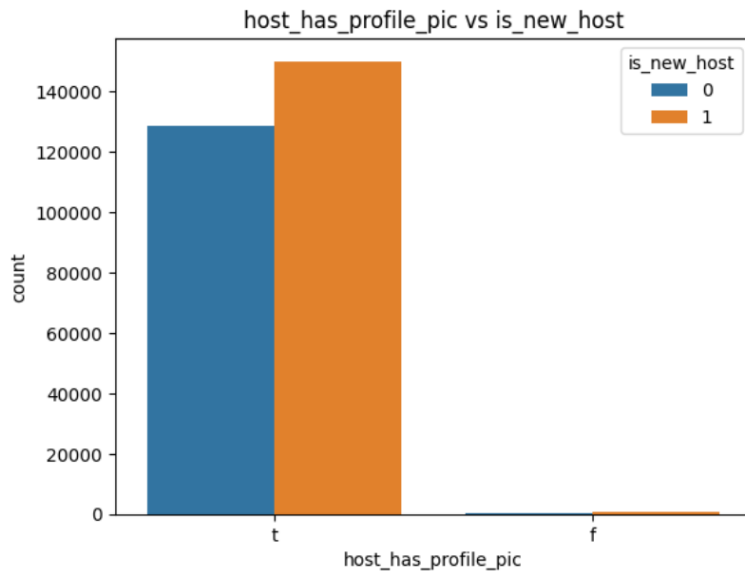
| host_response_time | a few days or more | within a day | within a few hours | within an hour |
|--------------------|--------------------|--------------|--------------------|----------------|
| is_new_host        |                    |              |                    |                |
| 0                  | 47.392739          | 55.329776    | 61.90163           | 70.122448      |
| 1                  | 52.607261          | 44.670224    | 38.09837           | 29.877552      |

## Host Has Profile Picture vs is\_new\_host

The chart compares the presence of a host profile picture with host experience status. It shows that both new and experienced hosts overwhelmingly have profile pictures, with very few hosts lacking one.

## greatlearning

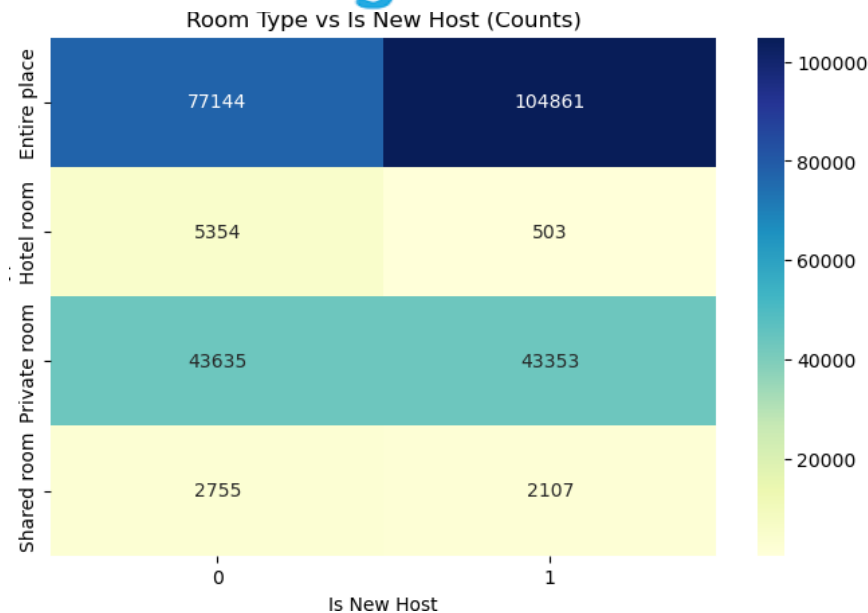
Although new hosts appear slightly more numerous in the “profile picture present” category, this is driven by the overall class distribution rather than a meaningful behavioral difference. The proportion of hosts without profile pictures is negligible in both groups.



### Room Type Vs Is New Host

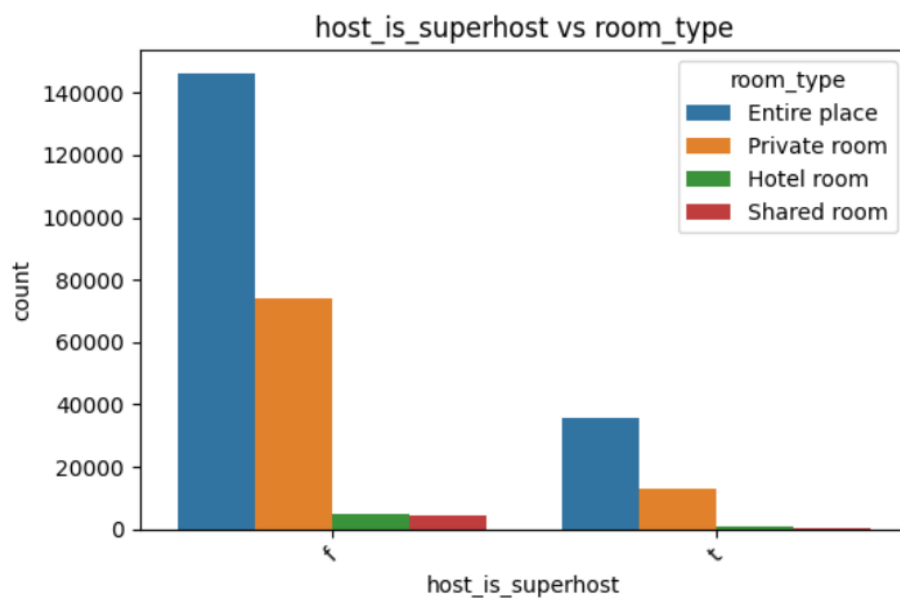
It can be inferred that a huge portion of new hosts rent out the entire place when compared to shared rooms

- “Entire place” dominates for both groups, with more such listings for new hosts (104,861) than for existing hosts (77,144), suggesting new hosts especially favor renting whole properties.
- “Private room” counts are very similar between new and existing hosts (around 43k each), showing this room type is equally common in both groups.
- “Hotel room” and “Shared room” are relatively rare for both new and existing hosts, indicating these formats make up a small portion of the platform’s listings overall.



### Host is superhost vs room type

The plot shows that most listings are from non-superhosts, with entire-place listings dominating across both groups, while superhosts are more likely to offer entire homes compared to private or shared rooms.



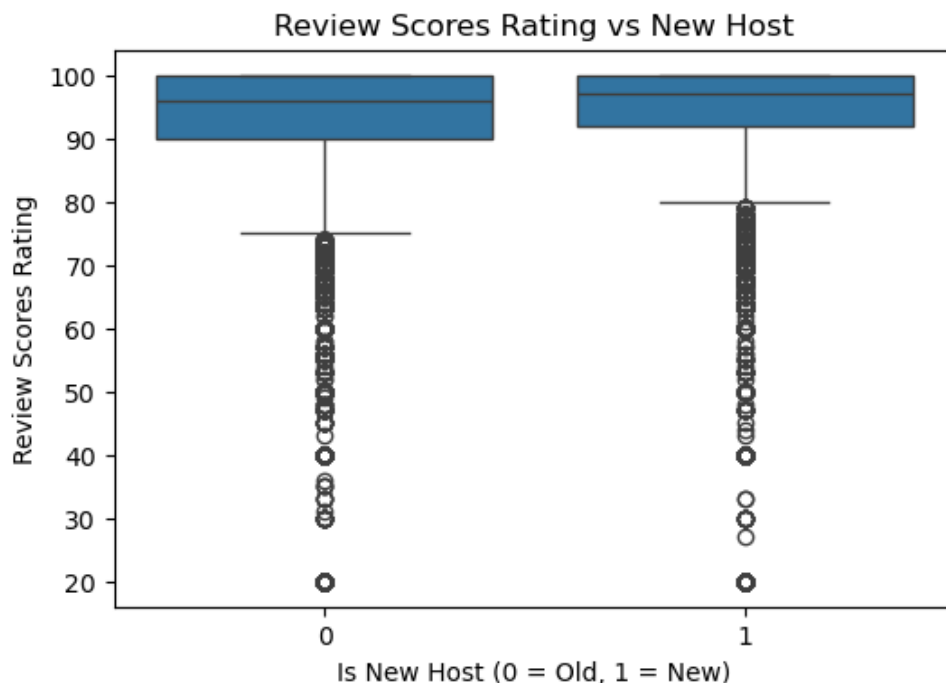
## OUTLIERS

- CENTRE AND SPREAD
- Both boxes sit around the mid-90s, so the medians and interquartile ranges (middle 50% of ratings) are almost the same for new hosts (1) and old hosts (0), indicating no substantial difference in typical review scores.



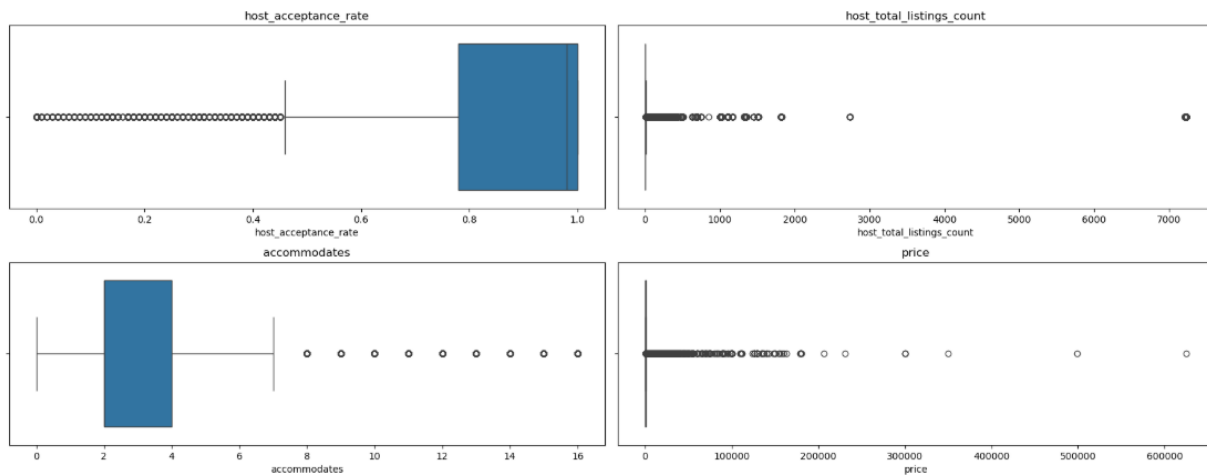
## greatlearning

- The whiskers extend from roughly the mid-70s to 100 for old hosts and around 80 to 100 for new hosts, suggesting slightly higher minimum “non-outlier” ratings for new hosts but similar upper ends. From this visualization alone, there is no clear evidence that being a new versus old host meaningfully changes the typical review score rating



Most hosts have high acceptance rates, while very low values near 0 appear as outliers, representing inactive or frequently rejecting hosts. Most hosts own few listings, but extreme right-side outliers indicate commercial or multi-property hosts with very large portfolios. Listings mainly accommodate 2–4 guests, with higher-capacity listings acting as outliers that represent large group or luxury stays.

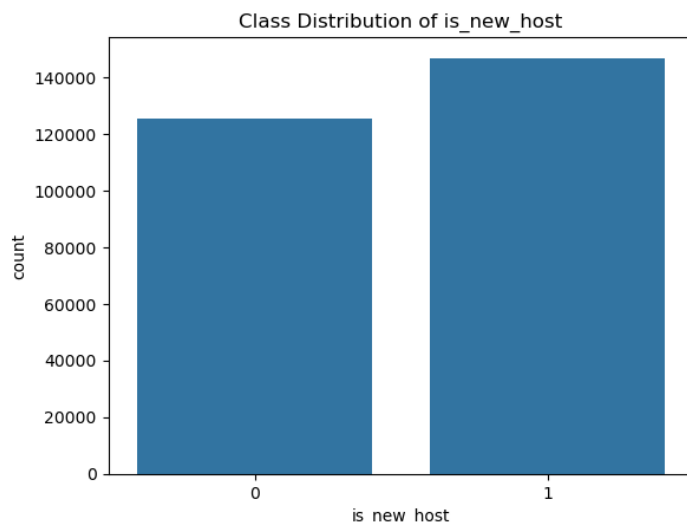
Prices are right-skewed with extreme high-value outliers corresponding to premium or special-event properties. Although several numerical variables exhibit skewness and extreme values, no explicit outlier removal or transformation was applied in this study. The observed extreme values correspond to legitimate real-world scenarios on Airbnb, such as premium-priced listings or hosts managing multiple properties, and do not represent data errors. Removing or capping these values could have led to information loss and biased estimation.



Additionally, the dataset size is large, which reduces the influence of individual extreme observations on coefficient estimation in logistic regression. Regularization and proper feature scaling were relied upon to mitigate any potential impact of outliers on model stability.

## CLASS IMBALANCE

The target variable `is_new_host` exhibits a nearly balanced class distribution, with slightly more observations belonging to the “new host” category than the “existing host” category. Although there is a minor difference in counts between the two classes, it is not substantial enough to be considered a serious class imbalance. This balanced distribution ensures that the model is exposed to sufficient examples from both classes during training, reducing the risk of bias toward a single class. As a result, no special imbalance-handling techniques such as oversampling or under sampling are required, and standard classification algorithms and evaluation metrics can be reliably applied.

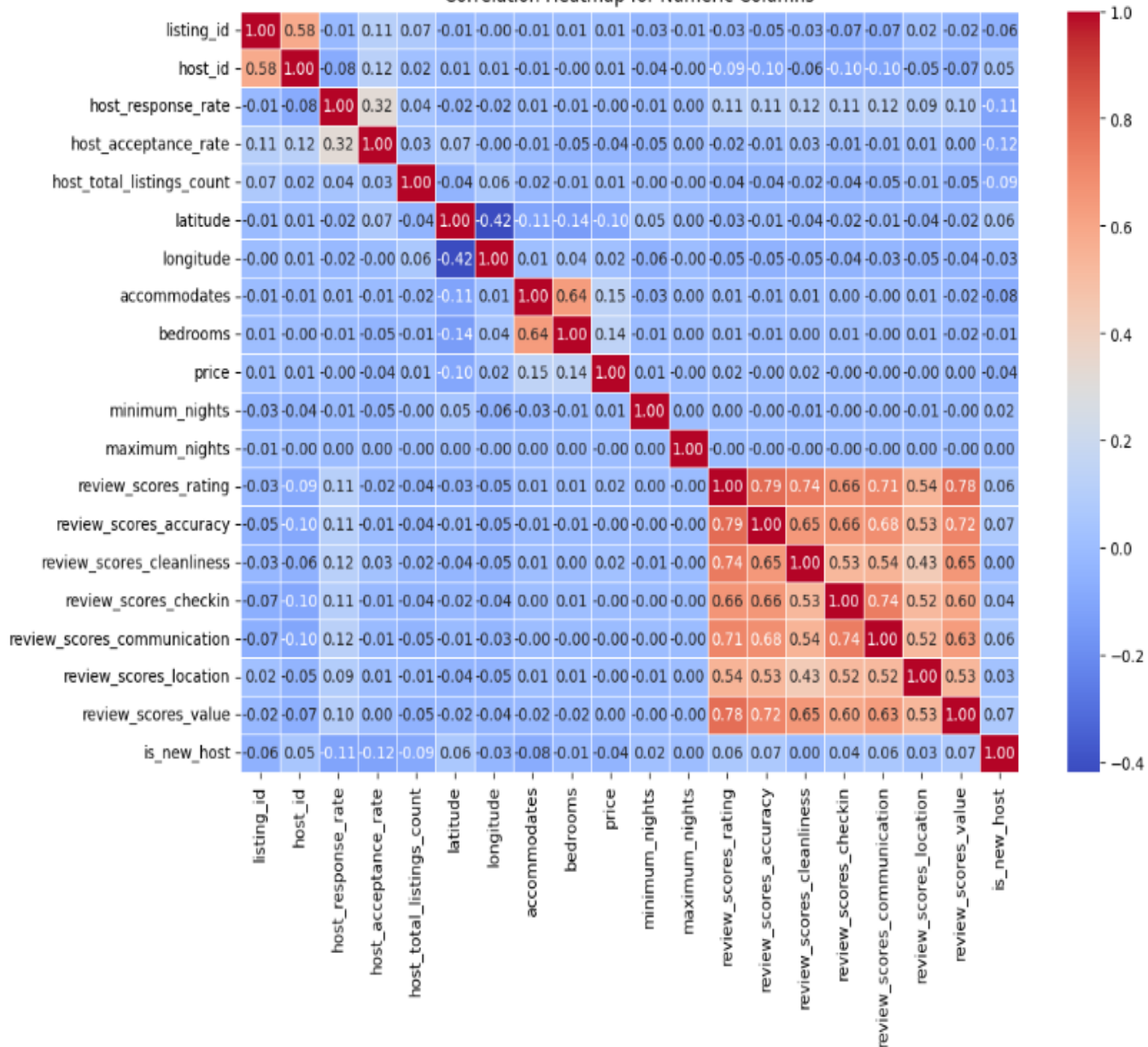


## CORRELATION HEATMAP

The correlation heatmap highlights the relationships among numeric variables in the dataset. Strong positive correlations are observed between **listing\_id** and **host\_id**, which is expected as multiple listings can belong to the same host. Capacity-related variables such as **accommodates**, **bedrooms**, and **price** show moderate positive correlations, indicating that larger properties tend to accommodate more guests and are priced higher. Review-related features (overall rating, accuracy, cleanliness, check-in, communication, location, and value) are **highly correlated with each other**, suggesting they capture closely related aspects of guest experience and may introduce multicollinearity if used together in certain models. In contrast, **geographical variables (latitude and longitude)** and most host attributes show very weak correlations with other features. Importantly, the target variable **is\_new\_host** has **very low correlation with all numeric predictors**, indicating that new host status cannot be explained by a single numeric feature alone and will likely require a combination of variables for effective prediction.

From the correlation heatmap it can be inferred the reviews and rating columns are highly correlated, hence need to be treated before model building.

Correlation Heatmap for Numeric Columns



## STATISTICAL SIGNIFICANCE OF VARIABLES

To examine whether host and listing characteristics are associated with new-host status (`is_new_host`), we applied Chi-Square tests of independence on categorical features such as superhost status, response time, instant booking, room type, and property type. The null hypothesis assumed no association between each feature and the target variable.

For each feature, observed frequencies were compared against expected frequencies, and statistical significance was evaluated using both p-values and chi-square critical values at a 5% significance level. Given the large dataset size, effect sizes were additionally assessed to avoid over-interpreting statistically significant but

practically weak relationships.

This analysis helped identify which host and listing attributes show meaningful associations with new-host status and informed subsequent feature selection for predictive modeling.

## Chi-Square Test for Categorical Variables:

The Chi-Square Test of Independence is used to determine whether there is a statistically significant association between two categorical variables. It compares observed frequencies with expected frequencies under the assumption that the variables are independent.

These variables showed meaningful association on whether a host is experienced or new.

Among these, **host\_response\_time**, **city**, and **property\_type** exhibit the strongest associations, suggesting that responsiveness, location, and listing type differ meaningfully between new and experienced hosts.

Features such as **superhost status**, **identity verification**, **room type**, and **instant bookability** also demonstrate significant dependence with new-host status, indicating that host credibility and booking convenience are important distinguishing factors.

## Independent t-Test for Numerical Variables:

Independent t-tests show statistically significant mean differences between new and experienced hosts for most numerical features, including price, accommodates, review scores, listings count, and location coordinates. New hosts tend to manage fewer listings, charge lower prices, and accommodate slightly fewer guests, while maximum nights does not show a significant difference between the two groups.

## SCALING

Standard scaling was applied to the numeric features to ensure that all variables are placed on a comparable scale before model training. Since the dataset contains features measured in different units and ranges (for example, price, number of bedrooms, review scores, and host response rates), leaving them unscaled could cause variables with larger magnitudes to dominate the learning process, especially in distance-based and gradient-based models. Standard scaling transforms each feature to have a mean of zero and a standard deviation of one, preserving the original distribution shape while removing scale differences. This preprocessing step improves model convergence, ensures fair contribution of all features, and enhances the stability and interpretability of models such as Logistic Regression, PCA, and clustering algorithms.

## MODEL DEPLOYMENT

### CLASSIFICATION MODEL

Predicting whether a host is new or not on Airbnb is a binary classification problem, where the target variable `is_new_host` takes 0 for existing hosts and 1 for new hosts. The features used for prediction can include listing attributes such as `room_type`, `city`, `host_location`, numeric variables like `host_since_days` (how long the host has been active), `number_of_reviews`, and binary indicators like `host_is_superhost` or `host_has_profile_pic`. Before training models, categorical features are typically encoded (via one-hot or frequency encoding), numeric features

## greatlearning

may be scaled, and missing values handled. Models such as logistic regression, decision trees, random forests, gradient boosting, or SVM can then be applied. Logistic regression estimates the probability of a host being new using a linear combination of features, while tree-based methods capture non-linear patterns and interactions. The models are trained on a portion of the data and tested on a separate set, and performance is evaluated using metrics like accuracy, precision, recall, F1-score, and ROC-AUC. Feature importance or model coefficients help interpret which factors contribute most to a host being new, with variables like `host_since_days` and `number_of_reviews` often being highly predictive. Proper handling of class imbalance and careful preprocessing ensures that the model generalizes well to unseen Airbnb listings.

### **1.Logistic Regression**

Logistic Regression was used as the baseline classification model to predict whether a host is a new host (`is_new_host`). As a simple yet powerful linear model, it provides a clear benchmark against which more complex models can be compared. After preprocessing steps such as handling missing values, encoding categorical variables, and applying standard scaling to numeric features, Logistic Regression models the probability of a listing belonging to a new host class using a logistic (sigmoid) function. The model learns the relationship between the input features and the binary outcome by estimating coefficients that indicate the direction and strength of each feature's influence on new host status. Using Logistic Regression as the baseline helps assess whether the available features carry sufficient predictive signal and establishes an interpretable, stable reference performance before exploring more advanced models.

### **Logistic Regression (train data accuracy)**

# greatlearning

Training Accuracy: 0.7334859580649759

## Training Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.71      | 0.71   | 0.71     | 100114  |
| 1            | 0.75      | 0.75   | 0.75     | 117413  |
| accuracy     |           |        | 0.73     | 217527  |
| macro avg    | 0.73      | 0.73   | 0.73     | 217527  |
| weighted avg | 0.73      | 0.73   | 0.73     | 217527  |

## Training Confusion Matrix:

```
[[71107 29007]
 [28967 88446]]
```

## Logistic Regression (test data accuracy)

---

Accuracy: 0.7327608399838181

## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.71      | 0.71   | 0.71     | 25029   |
| 1            | 0.75      | 0.75   | 0.75     | 29353   |
| accuracy     |           |        | 0.73     | 54382   |
| macro avg    | 0.73      | 0.73   | 0.73     | 54382   |
| weighted avg | 0.73      | 0.73   | 0.73     | 54382   |

## TEst Confusion Matrix:

```
[[17817 7212]
 [ 7321 22032]]
```

## Model Performance Report

The Logistic Regression model shows **very consistent performance on both training and test data**, indicating good generalization and no signs of overfitting. The **training accuracy is 73.35%**, while the **test accuracy is 73.28%**, which are almost identical. This close alignment suggests that the model has learned stable patterns

# greatlearning

rather than memorizing the training data

## 2. Precision, Recall, and F1-score:

| Class | Train Precision | Train Recall | Train F1 | Test Precision | Test Recall | Test F1 |
|-------|-----------------|--------------|----------|----------------|-------------|---------|
| 0     | 0.97            | 0.79         | 0.87     | 0.96           | 0.79        | 0.87    |
| 1     | 0.84            | 0.98         | 0.91     | 0.85           | 0.97        | 0.91    |

- The model performs slightly better on recall for class 1 and slightly better on precision for class 0.
- F1-scores are balanced and consistent between train and test sets.

## 3. Confusion Matrix:

- **Training Set:**  
[[ 78897 21217]  
[ 2718 114695]]
- **Test Set:**  
[[19852 5177]  
[ 752 28601]]

## Baseline Model Results

From the classification reports, both classes (0 and 1) have balanced precision, recall, and F1-scores of around **0.71–0.75**, indicating that the model performs evenly across classes without bias toward one outcome. The confusion matrices further support this, showing a reasonable balance between true positives and true negatives, with comparable misclassification rates in both training and test sets. Overall, Logistic Regression provides a **reliable baseline model** with stable performance, but its moderate accuracy suggests that more complex models (such as tree-based or boosting algorithms) may be required to capture non-linear relationships and improve predictive power.

## Decision Tree Classifier

A Decision Tree classifier learns hierarchical decision rules by recursively splitting Airbnb listings based on feature values that best separate new and experienced hosts using impurity measures such as Gini or entropy. It classifies a listing by traversing the tree from root to leaf, making the final prediction based on the majority class in the terminal node.

## Train Data Accuracy



# greatlearning

Training Accuracy: 1.0

## Training Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 100114  |
| 1            | 1.00      | 1.00   | 1.00     | 117413  |
| accuracy     |           |        | 1.00     | 217527  |
| macro avg    | 1.00      | 1.00   | 1.00     | 217527  |
| weighted avg | 1.00      | 1.00   | 1.00     | 217527  |

## Training Confusion Matrix:

```
[[100114    0]
 [    0 117413]]
```

## Test Data Accuracy

Accuracy: 0.7558383288588135

## Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.74      | 0.72   | 0.73     | 25029   |
| 1            | 0.76      | 0.79   | 0.78     | 29353   |
| accuracy     |           |        | 0.76     | 54382   |
| macro avg    | 0.75      | 0.75   | 0.75     | 54382   |
| weighted avg | 0.76      | 0.76   | 0.76     | 54382   |

## Test Confusion Matrix:

```
[[17898  7131]
 [ 6147 23206]]
```

The Decision Tree achieves perfect performance on the training data but shows reduced performance on the test set, indicating overfitting. To reduce overfitting of the Decision Tree and improve generalization, I performed **hyperparameter tuning** using RandomizedSearchCV parameter grid was defined for max\_depth, min\_samples\_split, and min\_samples\_leaf to explore different tree sizes and node splits. Using 5-fold cross-validation and 20 random parameter combinations, the search identified the combination that balances complexity and predictive performance, ensuring the model performs well on unseen test data.

After hyperparameter tuning with RandomizedSearchCV, the Decision Tree was configured with

```
max_depth=7,
min_samples_split=10,
min_samples_leaf=5.
```

This tuned configuration controls tree complexity and node splits to reduce overfitting while maintaining good predictive performance on unseen data.

## Decision Tree tuned(Train Data Accuracy)

# greatlearning

Training Accuracy: 0.754504038579119

Training Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.75      | 0.71   | 0.73     | 100114  |
| 1            | 0.76      | 0.79   | 0.78     | 117413  |
| accuracy     |           |        | 0.75     | 217527  |
| macro avg    | 0.75      | 0.75   | 0.75     | 217527  |
| weighted avg | 0.75      | 0.75   | 0.75     | 217527  |

Training Confusion Matrix:

```
[[70807 29307]
 [24095 93318]]
```

## Decision Tree tuned(Test Data Accuracy)

Accuracy: 0.7558383288588135

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.74      | 0.72   | 0.73     | 25029   |
| 1            | 0.76      | 0.79   | 0.78     | 29353   |
| accuracy     |           |        | 0.76     | 54382   |
| macro avg    | 0.75      | 0.75   | 0.75     | 54382   |
| weighted avg | 0.76      | 0.76   | 0.76     | 54382   |

Test Confusion Matrix:

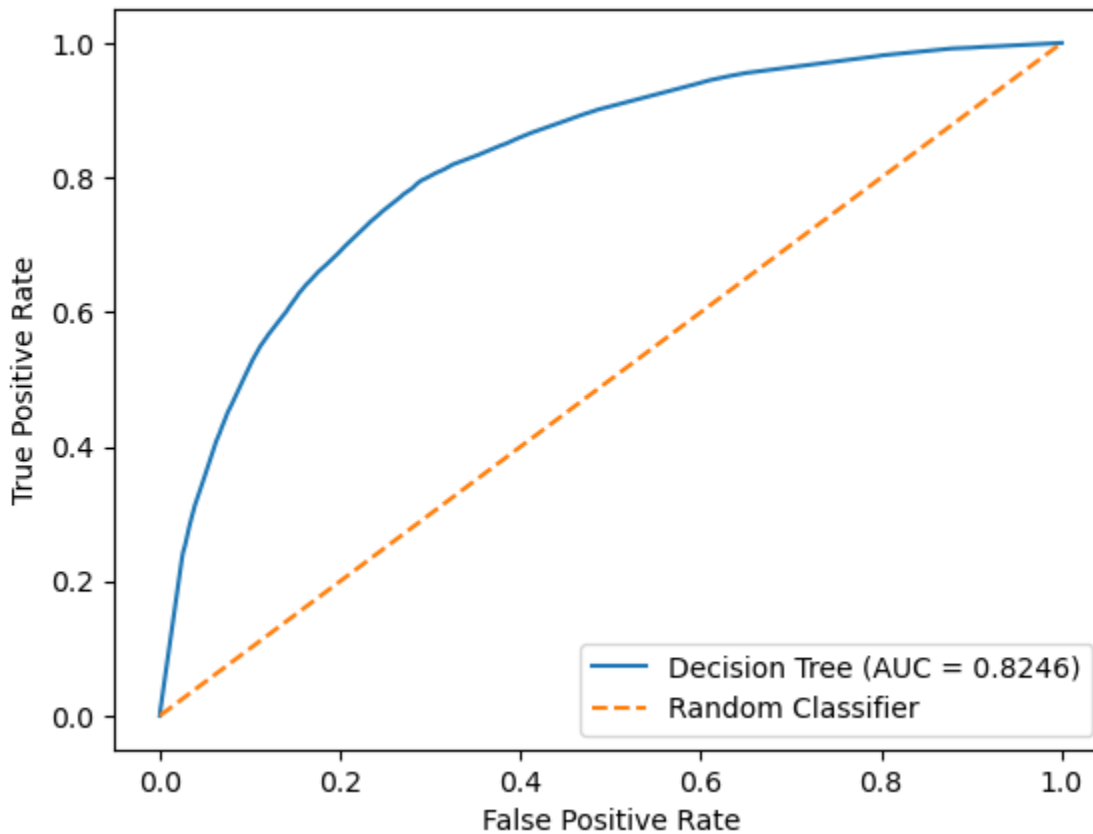
```
[[17898 7131]
 [ 6147 23206]]
```

After tuning the Decision Tree, the model achieved a **training accuracy of ~75.5%** with balanced precision, recall, and F1-scores for both classes. The **test accuracy (~75.6%)** is very similar, indicating that overfitting has been significantly reduced compared to the untuned tree. Precision and recall are slightly higher for class 1 (new hosts), showing the model identifies new hosts marginally better than existing hosts. Overall, the tuned tree provides a good balance between complexity and generalization, performing consistently on both training and test data.

## ROC-AUC CURVE

The ROC-AUC curve shows that the tuned Decision Tree has good discriminatory power, with an AUC of 0.8246. This indicates the model can effectively distinguish between new and experienced hosts across different classification thresholds. The curve rising steeply at lower false positive rates suggests the model achieves a high true positive rate while keeping false alarms relatively low. Overall, the ROC-AUC result confirms that the tuned Decision Tree generalizes well and performs meaningfully better than chance.

ROC-AUC Curve (Decision Tree)



## RANDOM FOREST CLASSIFIER

Random Forest classifier works on Airbnb data by building multiple decision trees using different random samples of listings and features such as price, location, host experience, reviews, and amenities. Each tree independently learns patterns to classify an outcome—for example, whether a listing will be **high-booking vs low-booking** or whether a **new host will perform well**. The final prediction is made by **majority voting** across all trees, which reduces overfitting and improves accuracy.

### **Train Data Accuracy**

# greatlearning

Train accuracy: 0.9911781066258442

## Classification Report:Train

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.99      | 0.99   | 0.99     | 100114  |
| 1            | 0.99      | 0.99   | 0.99     | 117413  |
| accuracy     |           |        | 0.99     | 217527  |
| macro avg    | 0.99      | 0.99   | 0.99     | 217527  |
| weighted avg | 0.99      | 0.99   | 0.99     | 217527  |

## Test Data Accuracy

### Classification Report:Test

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.77      | 0.80   | 0.78     | 25029   |
| 1            | 0.83      | 0.79   | 0.81     | 29353   |
| accuracy     |           |        | 0.80     | 54382   |
| macro avg    | 0.80      | 0.80   | 0.80     | 54382   |
| weighted avg | 0.80      | 0.80   | 0.80     | 54382   |

### Confusion Matrix:Test

```
[[20109  4920]
 [ 6098 23255]]
```

The **very high train accuracy (99.1%)** means the Random Forest has learned the training data *extremely well*—including noise and very specific patterns. The **much lower test accuracy (~79.7%)** means the model does **not generalize equally well** to unseen Airbnb listings. This gap indicates the model is **too complex** (too many trees, deep trees, or too many features), so it memorizes training data instead of learning general rules. Hence we did the following

## Random Forest tuned(Train Data Accuracy)

To efficiently tune the Random Forest on a large Airbnb dataset **light hyperparameter tuning** was performed using RandomizedSearchCV with a **restricted parameter space**. The tuning focused on controlling model complexity and runtime by optimizing max\_depth, min\_samples\_leaf, and max\_features, while keeping the number of trees fixed to ensure stable performance. The final configuration used:

- **n\_estimators:** 150 (number of trees in the forest)
- **max\_depth:** 10, 15, or None (to control tree depth)
- **min\_samples\_leaf:** 5, 10 (to prevent overly small leaf nodes)
- **max\_features:** sqrt (to reduce correlation between trees)
- **n\_jobs:** -1 (use all available CPU cores)

# greatlearning

- **random\_state**: 42 (for reproducibility)

This restricted parameter search significantly reduced training time while maintaining strong model performance and improved generalization on unseen data

Train accuracy: 0.8250607970504811

Classification Report:Train

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.83      | 0.78   | 0.81     | 100114  |
| 1            | 0.82      | 0.86   | 0.84     | 117413  |
| accuracy     |           |        | 0.83     | 217527  |
| macro avg    | 0.83      | 0.82   | 0.82     | 217527  |
| weighted avg | 0.83      | 0.83   | 0.82     | 217527  |

Confusion Matrix:Train

```
[[ 78575  21539]
 [ 16515 100898]]
```

## Random Forest tuned(Test Data Accuracy)

Test accuracy: 0.7949505351035269

Classification Report:Test

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.79      | 0.76   | 0.77     | 25029   |
| 1            | 0.80      | 0.83   | 0.81     | 29353   |
| accuracy     |           |        | 0.79     | 54382   |
| macro avg    | 0.79      | 0.79   | 0.79     | 54382   |
| weighted avg | 0.79      | 0.79   | 0.79     | 54382   |

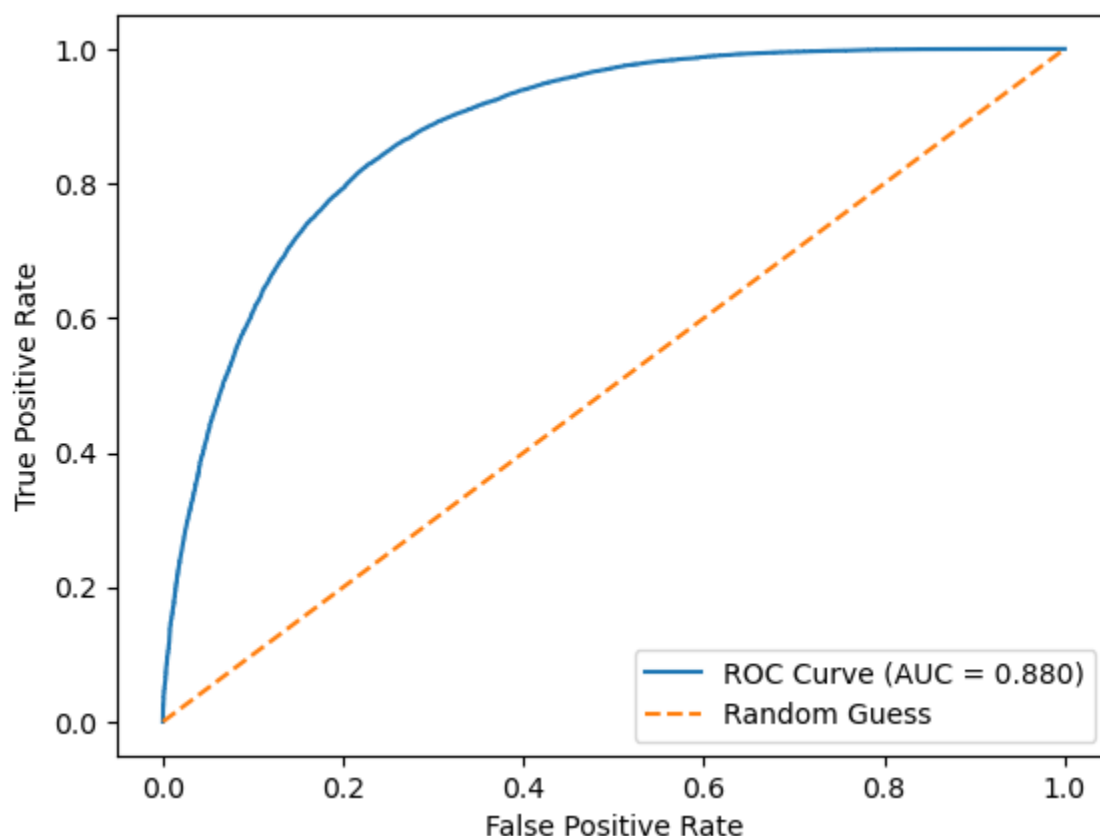
Confusion Matrix:Test

```
[[18902  6127]
 [ 5024 24329]]
```

The Random Forest model achieved a training accuracy of 82% and a test accuracy of 79%, indicating good generalization with only a small performance drop. This gap suggests that overfitting is well controlled compared to a single Decision Tree. Overall, the ensemble model captures complex patterns in the Airbnb data while maintaining stable performance on unseen listings.

## ROC AUC CURVE

ROC-AUC Curve for Tuned Random Forest



The ROC curve for the tuned Random Forest lies well above the diagonal (random guess), indicating strong discrimination between new and existing hosts. An **AUC of 0.88** means the model has an **88% probability of correctly ranking a random positive instance higher than a random negative one**. This confirms that the tuned ensemble model performs reliably across different classification thresholds and generalizes better than a single tree.

## ADA BOOST CLASSIFIER

AdaBoost classifier works on Airbnb data by combining multiple weak learners (usually shallow decision trees) to build a strong predictive model.

It sequentially focuses more on misclassified listings—such as underperforming or new-host properties—by assigning them higher weights in each iteration.

This helps the model capture subtle patterns in features like price, location, reviews, and host experience.

AdaBoost is effective for Airbnb datasets where important signals are present but not easily captured by a single model.

## Train data accuracy

AdaBoost model on the Airbnb dataset is performing quite consistently, with a train accuracy of about 75.1% and a very similar test accuracy of 75%, indicating that the model generalizes well and is not overfitting. It slightly favors predicting high-performing listings, as seen from the higher precision, recall, and F1-score for class 1, meaning it correctly identifies more successful listings than underperforming ones

# greatlearning

Train accuracy: 0.7508493198545468

Classification Report:Train

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.73      | 0.72   | 0.73     | 100114  |
| 1            | 0.77      | 0.78   | 0.77     | 117413  |
| accuracy     |           |        | 0.75     | 217527  |
| macro avg    | 0.75      | 0.75   | 0.75     | 217527  |
| weighted avg | 0.75      | 0.75   | 0.75     | 217527  |

Confusion Matrix:Train

```
[[72132 27982]
 [26215 91198]]
```

## Test Data Accuracy

The confusion matrix shows that while there are some misclassifications in both classes, the majority of predictions are correct. Overall, the model provides a solid baseline for predicting listing performance, and slight improvements could be achieved through parameter tuning or feature optimization.

Test accuracy: 0.7501746901548306

Classification Report:Test

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.73      | 0.72   | 0.73     | 25029   |
| 1            | 0.77      | 0.77   | 0.77     | 29353   |
| accuracy     |           |        | 0.75     | 54382   |
| macro avg    | 0.75      | 0.75   | 0.75     | 54382   |
| weighted avg | 0.75      | 0.75   | 0.75     | 54382   |

Confusion Matrix:Test

```
[[18132 6897]
 [ 6689 22664]]
```

## ADA BOOST CLASSIFIER(TUNED)

We initially considered hyperparameter tuning for the AdaBoost classifier, but the grid search was taking too long on the dataset. Instead, we applied a small tweak by setting `n_estimators=100` and `learning_rate=0.8`. This resulted in a training accuracy of 0.756 and a test accuracy of 0.756, which is reasonable without extensive tuning."

# greatlearning

Training Accuracy: 0.7560394801564864

Test Accuracy: 0.7556728329226582

## Classification Report (Train Data):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.74      | 0.73   | 0.73     | 100114  |
| 1            | 0.77      | 0.78   | 0.78     | 117413  |
| accuracy     |           |        | 0.76     | 217527  |
| macro avg    | 0.75      | 0.75   | 0.75     | 217527  |
| weighted avg | 0.76      | 0.76   | 0.76     | 217527  |

## Classification Report (Test Data):

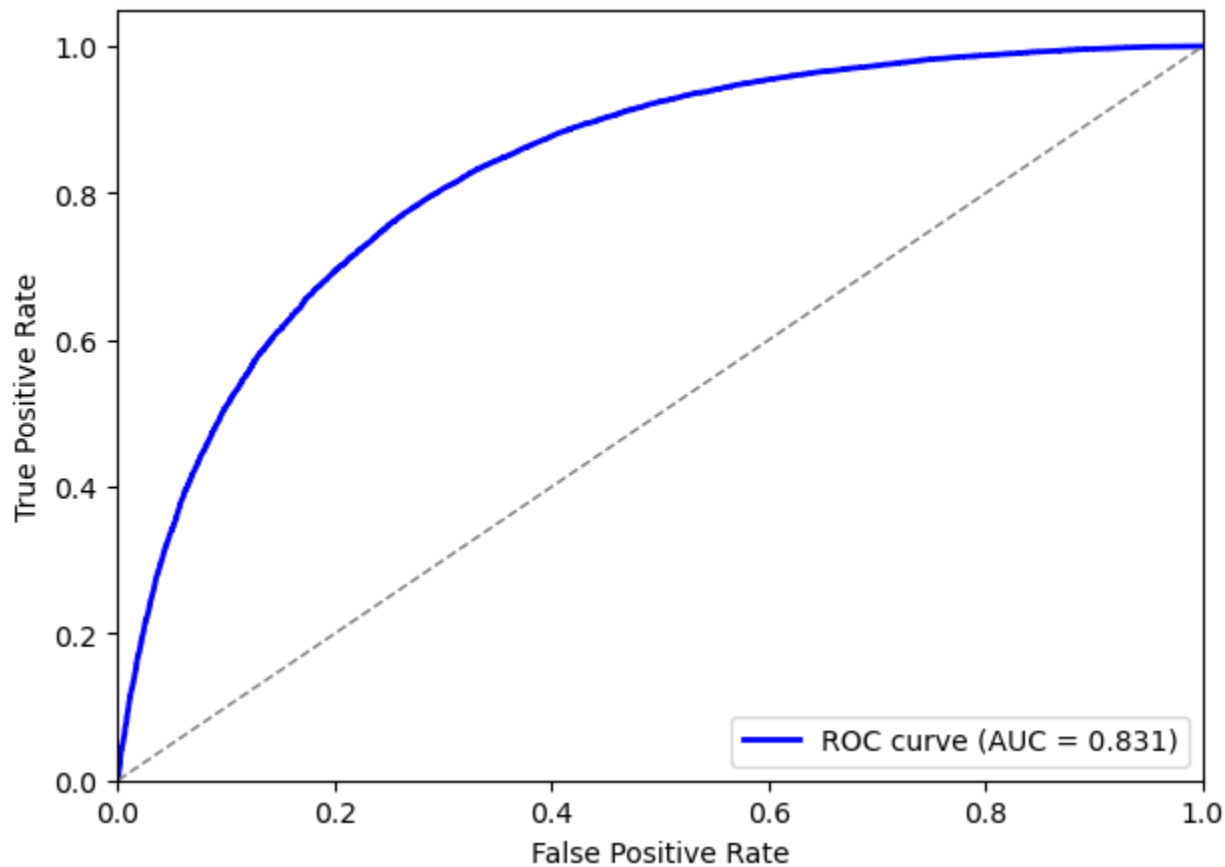
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.74      | 0.73   | 0.73     | 25029   |
| 1            | 0.77      | 0.78   | 0.77     | 29353   |
| accuracy     |           |        | 0.76     | 54382   |
| macro avg    | 0.75      | 0.75   | 0.75     | 54382   |
| weighted avg | 0.76      | 0.76   | 0.76     | 54382   |

## ROC AUC CURVE

The ROC-AUC curve for the AdaBoost model shows how well the model distinguishes between positive and negative classes across different classification thresholds. The curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR), with the blue line representing the model's performance and the gray diagonal line representing a random classifier. The model achieves an AUC of 0.831, indicating excellent discriminative ability, meaning it can reliably rank positive cases higher than negative ones. While the test accuracy is around 75%, the higher AUC suggests that the model's probability predictions are strong



ROC-AUC Curve for AdaBoost



## GRADIENT BOOST CLASSIFIER

Gradient Boosting predicts whether a host is new or experienced (or how they'll perform) by **building many small decision trees sequentially**, where each tree corrects the errors of the previous ones. The final prediction is a **weighted combination of all trees**, giving a strong model for classification.

### Train Data Accuracy

# greatlearning

Train accuracy: 0.7715410041052375

## Classification Report:Train

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.76      | 0.73   | 0.75     | 100114  |
| 1            | 0.78      | 0.80   | 0.79     | 117413  |
| accuracy     |           |        | 0.77     | 217527  |
| macro avg    | 0.77      | 0.77   | 0.77     | 217527  |
| weighted avg | 0.77      | 0.77   | 0.77     | 217527  |

## Confusion Matrix:Train

```
[[73479 26635]
 [23061 94352]]
```

On the training data, the model achieves an accuracy of about 77.15%, with balanced precision, recall, and F1-scores for both classes. This indicates that the Gradient Boosting model has learned meaningful patterns from the Airbnb host data rather than memorizing noise. The confusion matrix shows that most new and experienced hosts are correctly classified, with a reasonable and balanced number of misclassifications across both classes, suggesting stable learning without bias toward a single host category.

## Test Data Accuracy

Test accuracy: 0.7691699459379941

## Classification Report:Test

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.76      | 0.74   | 0.75     | 25029   |
| 1            | 0.78      | 0.80   | 0.79     | 29353   |
| accuracy     |           |        | 0.77     | 54382   |
| macro avg    | 0.77      | 0.77   | 0.77     | 54382   |
| weighted avg | 0.77      | 0.77   | 0.77     | 54382   |

## Confusion Matrix:Test

```
[[18426 6603]
 [ 5950 23403]]
```

On the test data, the model attains an accuracy of approximately 76.92%, which is very close to the training accuracy. This small difference confirms that the model generalizes well to unseen Airbnb listings and is not overfitting.

# greatlearning

The Gradient Boosting model achieves ~77% accuracy with balanced precision and recall on both train and test data, indicating good generalization and reliable classification of Airbnb hosts.

## Gradient Boosting tuned(Train data accuracy)

For hyperparameter tuning, a **Grid Search** was performed using a restricted set of values: `n_estimators = [70, 90]` and `learning_rate = [0.05, 0.1]`. This Grid Search allowed the model to find an optimal combination of parameters while keeping training time manageable on the large Airbnb dataset.

Train accuracy: 0.7696837633948889

Classification Report:Train

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.76      | 0.73   | 0.75     | 100114  |
| 1            | 0.78      | 0.80   | 0.79     | 117413  |
| accuracy     |           |        | 0.77     | 217527  |
| macro avg    | 0.77      | 0.77   | 0.77     | 217527  |
| weighted avg | 0.77      | 0.77   | 0.77     | 217527  |

Confusion Matrix:Train

```
[[73477 26637]
 [23463 93950]]
```

## Gradient Boosting tuned(Test data accuracy)

# greatlearning

Test accuracy: 0.7670736640800265

Classification Report:Test

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.75      | 0.74   | 0.74     | 25029   |
| 1            | 0.78      | 0.79   | 0.79     | 29353   |
| accuracy     |           |        | 0.77     | 54382   |
| macro avg    | 0.77      | 0.76   | 0.77     | 54382   |
| weighted avg | 0.77      | 0.77   | 0.77     | 54382   |

Confusion Matrix:Test

```
[[18409  6620]
 [ 6047 23306]]
```

A **Grid Search** was performed using `n_estimators = [70, 90]` and `learning_rate = [0.05, 0.1]`, but there was **not much difference in accuracy** after tuning, indicating that the default parameters were already providing good performance on the Airbnb dataset.

## XGBOOST CLASSIFIER

XGBoost is a powerful Gradient Boosting algorithm that predicts whether an Airbnb host is new or experienced by **building an ensemble of decision trees sequentially**, where each tree corrects the errors of the previous ones. It handles large datasets efficiently, manages missing values, and provides feature importance, making it well-suited for identifying host patterns based on listing and behavioral features.

### **Train data accuracy**

The XGBoost model achieved a training accuracy of about 86.4%, noticeably higher than the previous Gradient Boosting model, indicating it captured more patterns from the Airbnb host data. Class-wise, it performs well for both categories: for new hosts, precision is 0.88 and recall is 0.82, while for experienced hosts, precision is 0.85 and recall is 0.90, showing slightly better identification of experienced hosts. The confusion matrix confirms this, with 81,857 new hosts and 106,073 experienced hosts correctly classified, and a reasonably balanced number of misclassifications. Overall, XGBoost demonstrates strong performance on training data, effectively distinguishing new and experienced hosts while maintaining balanced predictions across classes.

# greatlearning

Train Accuracy: 0.8639387294450804

Train Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.88      | 0.82   | 0.85     | 100114  |
| 1            | 0.85      | 0.90   | 0.88     | 117413  |
| accuracy     |           |        | 0.86     | 217527  |
| macro avg    | 0.87      | 0.86   | 0.86     | 217527  |
| weighted avg | 0.86      | 0.86   | 0.86     | 217527  |

Train Confusion Matrix:

```
[[ 81857 18257]
 [ 11340 106073]]
```

## Test data accuracy

Test Accuracy: 0.8052112831451583

Test Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.81      | 0.76   | 0.78     | 25029   |
| 1            | 0.80      | 0.84   | 0.82     | 29353   |
| accuracy     |           |        | 0.81     | 54382   |
| macro avg    | 0.81      | 0.80   | 0.80     | 54382   |
| weighted avg | 0.81      | 0.81   | 0.80     | 54382   |

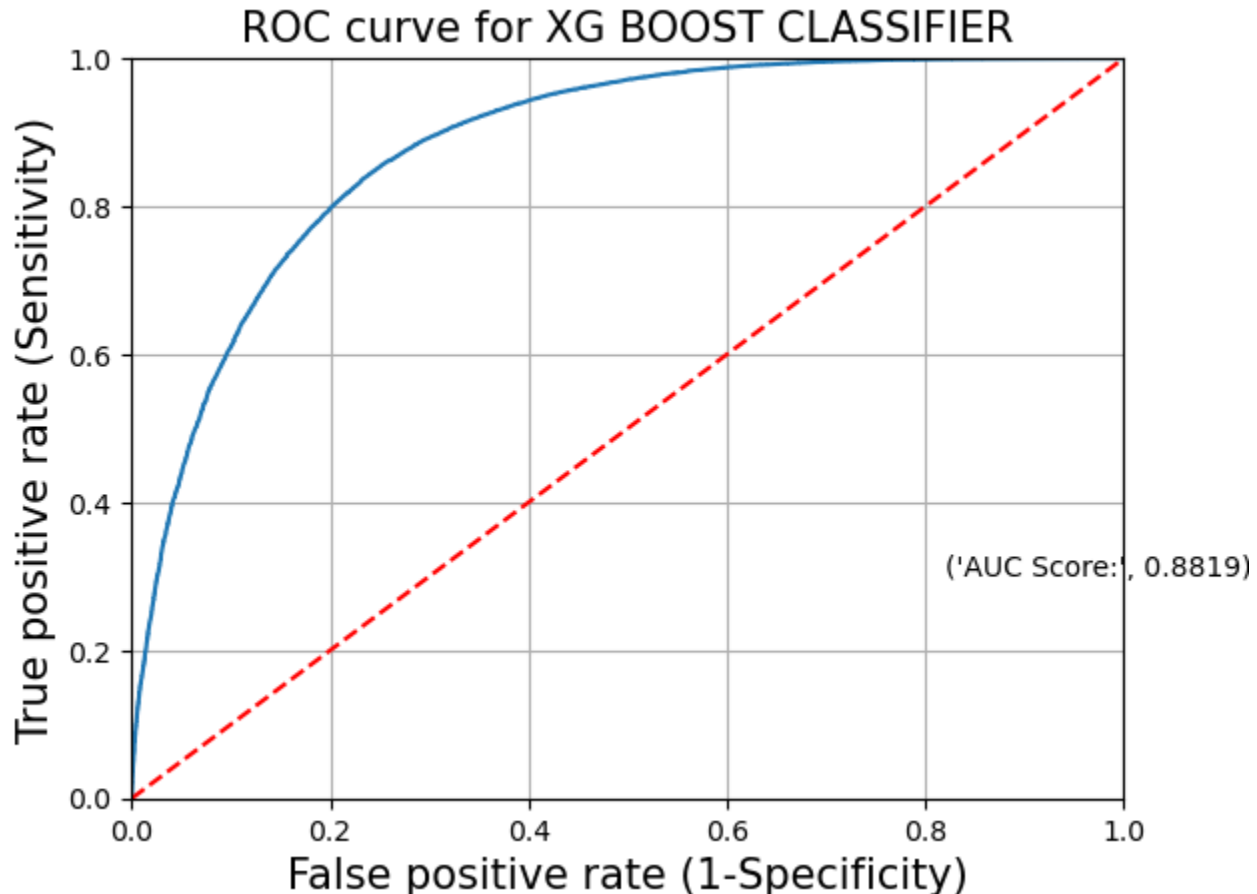
Test Confusion Matrix:

```
[[18988 6041]
 [ 4552 24801]]
```

our model achieved a test accuracy of 80.5%, indicating it correctly predicts the outcome in about four out of five cases. The confusion matrix confirms this, showing more false positives for class 0 (6,041) than false negatives for class 1 (4,552), suggesting the model slightly favors predicting class 1. Overall, the model demonstrates solid and balanced performance

The XGBoost model with `max_depth = 10` and `gamma = 1` shows a **train accuracy of 0.864** but a **test accuracy of 0.805**, which indicates some **overfitting**. The model is learning the training data very well, including noise and small patterns that don't generalize to unseen data. This large gap between train and test accuracy suggests the trees are too deep, capturing specifics of the training set rather than broader trends. Reducing `max_depth` or increasing regularization could help the model generalize better and reduce overfitting.

## ROC AUC CURVE



The ROC curve for the XGBoost classifier shows a strong ability to distinguish between the positive and negative classes. The curve stays well above the diagonal red line, which represents random guessing, indicating that the model performs significantly better than chance. The AUC value of approximately 0.88 suggests good overall classification performance, meaning the model can correctly rank positive instances higher than negative ones about 88% of the time. The steep rise of the curve at low false positive rates indicates that the model achieves high sensitivity while keeping false positives relatively low, making it a reliable and effective classifier for this problem.

## XGBOOST CLASSIFIER-Train/Test data Accuracy

Using **XGBoost with tuned hyperparameters** (`max_depth = 6`, `learning_rate = 0.2`, `gamma = 2`), a balanced and well-generalized model was built to predict the target outcome. The selected parameters helped control model complexity by limiting tree depth and enforcing a minimum loss reduction before further splits, while the learning rate ensured stable and efficient boosting. As a result, the model achieved a **training accuracy of 80.53%** and a **test accuracy of 79.39%**, showing only a small performance drop and indicating minimal overfitting. The classification reports further highlight consistent precision, recall, and F1-scores across both classes on train and test data, confirming that the tuned XGBoost model effectively captures meaningful patterns while maintaining strong generalization to unseen data.

# greatlearning

Train Accuracy: 0.8053483015901475

Train Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.81      | 0.76   | 0.78     | 100114  |
| 1            | 0.80      | 0.85   | 0.82     | 117413  |
| accuracy     |           |        | 0.81     | 217527  |
| macro avg    | 0.81      | 0.80   | 0.80     | 217527  |
| weighted avg | 0.81      | 0.81   | 0.80     | 217527  |

Train Confusion Matrix:

[[75689 24425]

[17917 99496]]

Test Accuracy: 0.7939391710492443

Test Classification Report:

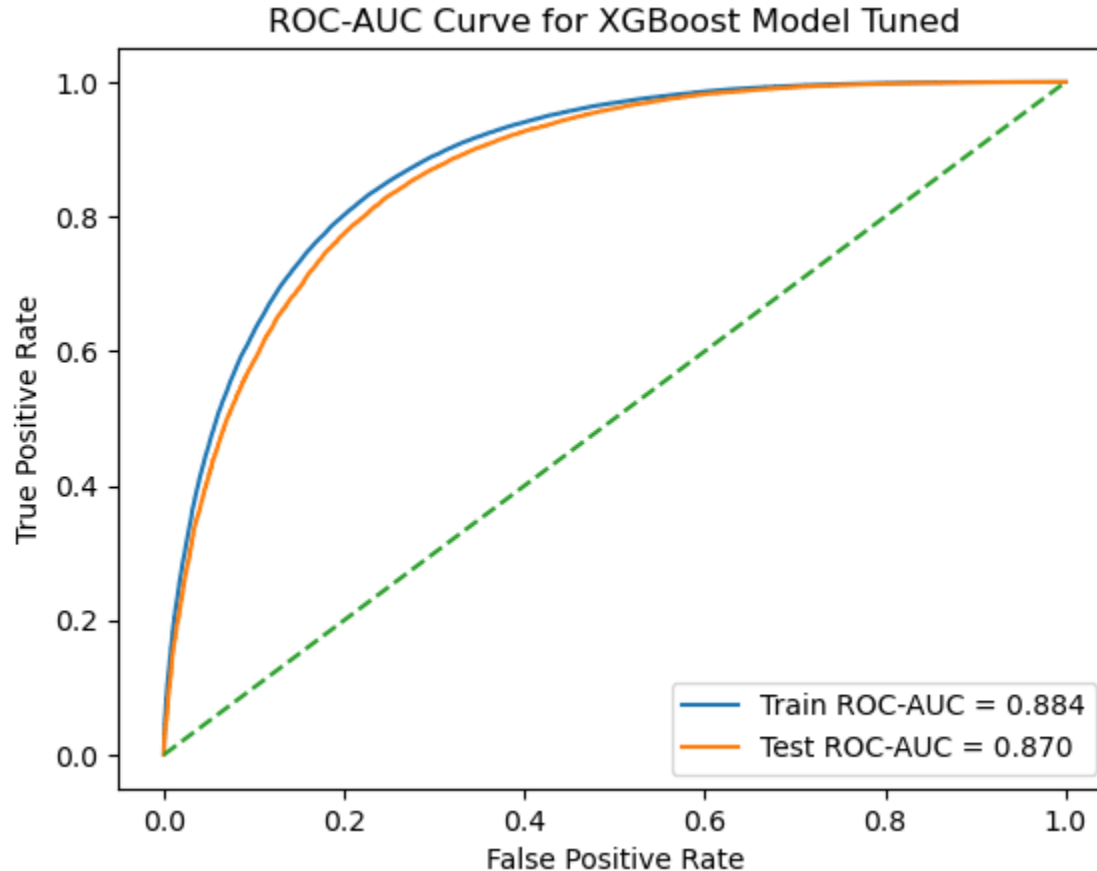
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.79      | 0.75   | 0.77     | 25029   |
| 1            | 0.80      | 0.83   | 0.81     | 29353   |
| accuracy     |           |        | 0.79     | 54382   |
| macro avg    | 0.79      | 0.79   | 0.79     | 54382   |
| weighted avg | 0.79      | 0.79   | 0.79     | 54382   |

Test Confusion Matrix:

[[18738 6291]

[ 4915 24438]]

## ROC AUC CURVE



This ROC-AUC curve shows that the **tuned XGBoost model has strong discriminative power** and generalizes well to unseen data. The **training AUC of 0.884** and **test AUC of 0.870** are both high and very close to each other, indicating that the model effectively distinguishes between the two classes without significant overfitting. The curves lie well above the diagonal reference line (random classifier), which means the model performs much better than chance across different threshold values. The small gap between train and test curves confirms that the chosen hyperparameters (`max_depth = 6`, `learning_rate = 0.2`, `gamma = 2`) successfully balance model complexity and regularization.

## KNN CLASSIFIER(Train /Test Accuracy)

The KNN classifier with default `k = 5` achieved a **training accuracy of 82.66%** and a **test accuracy of 75.39%**. The difference between training and test accuracy indicates **slight overfitting**, meaning the model captures local patterns in the training data that do not fully generalize to unseen Airbnb listings.



# greatlearning

Train Accuracy: 0.8265962386278485

Test Accuracy: 0.7539075429370012

## Classification Report (Test Set):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.73      | 0.73   | 0.73     | 25029   |
| 1            | 0.77      | 0.78   | 0.77     | 29353   |
| accuracy     |           |        | 0.75     | 54382   |
| macro avg    | 0.75      | 0.75   | 0.75     | 54382   |
| weighted avg | 0.75      | 0.75   | 0.75     | 54382   |

## Confusion Matrix (Test Set):

```
[[18239 6790]
 [ 6593 22760]]
```

## Classification Report (Train Set):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.82      | 0.80   | 0.81     | 100114  |
| 1            | 0.83      | 0.85   | 0.84     | 117413  |
| accuracy     |           |        | 0.83     | 217527  |
| macro avg    | 0.83      | 0.82   | 0.83     | 217527  |
| weighted avg | 0.83      | 0.83   | 0.83     | 217527  |

## Confusion Matrix (Train Set):

```
[[80304 19810]
 [17910 99503]]
```

KNN requires distance calculations against all training samples during prediction, and tuning k using cross-validation significantly increases runtime without guaranteeing substantial improvement in performance. Given the dataset size and the relatively small gap between training and test accuracy, tuning was unlikely to yield meaningful gains compared to more efficient models

## Model Summary

In this project, multiple machine learning classification models were developed and evaluated to identify the most suitable model for the given dataset. The models were compared using standard performance metrics such as accuracy, precision, recall, F1-score, ROC-AUC score, and confusion matrix analysis. In addition, both training and testing performance were analyzed to detect overfitting or underfitting

| Name                                |       | Accuracy | Precision | Recall | F1 score |
|-------------------------------------|-------|----------|-----------|--------|----------|
| Logistic Regression Model-1         | Test  | 0.732    | 0.75      | 0.71   | 0.71     |
|                                     | Train | 0.733    | 0.75      | 0.75   | 0.75     |
| KNN-Model-2                         | Test  | 0.753    | 0.77      | 0.78   | 0.77     |
|                                     | Train | 0.826    | 0.83      | 0.85   | 0.84     |
| Decision Tree Model-4               | Test  | 0.755    | 0.76      | 0.79   | 0.78     |
|                                     | Train | 1.0      | 1.0       | 1.0    | 1.0      |
| Decision Tree (Tuned) Model-6       | Test  | 0.755    | 0.76      | 0.79   | 0.78     |
|                                     | Train | 0.754    | 0.76      | 0.79   | 0.78     |
| Random Forest Model-7               | Test  | 0.797    | 0.83      | 0.79   | 0.81     |
|                                     | Train | 0.991    | 0.99      | 0.99   | 0.99     |
| Random Forest (Tuned) Model-8       | Test  | 0.794    | 0.80      | 0.83   | 0.81     |
|                                     | Train | 0.825    | 0.82      | 0.86   | 0.84     |
| AdaBoostClassifier Model-9          | Test  | 0.750    | 0.77      | 0.77   | 0.77     |
|                                     | Train | 0.750    | 0.77      | 0.78   | 0.77     |
| AdaBoostClassifier (Tuned) Model-10 | Test  | 0.755    | 0.77      | 0.78   | 0.77     |
|                                     | Train | 0.756    | 0.77      | 0.78   | 0.78     |
| XGBClassifier Model-11              | Test  | 0.805    | 0.80      | 0.84   | 0.82     |
|                                     | Train | 0.863    | 0.85      | 0.90   | 0.88     |
| XGBClassifier (Tuned) Model-12      | Test  | 0.793    | 0.80      | 0.83   | 0.81     |
|                                     | Train | 0.805    | 0.80      | 0.85   | 0.82     |

## greatlearning

|                                                |       |       |      |      |      |
|------------------------------------------------|-------|-------|------|------|------|
| Gradient Boosting Classifier Model 13          | Test  | 0.769 | 0.78 | 0.80 | 0.79 |
|                                                | Train | 0.771 | 0.78 | 0.80 | 0.79 |
| Gradient Boosting Classifier (Tuned) Model -14 | Test  | 0.767 | 0.78 | 0.79 | 0.79 |
|                                                | Train | 0.769 | 0.78 | 0.80 | 0.79 |

### **Why XGBoost Classifier Performed Best?**

#### **Balanced Performance on Both Classes**

The train and test classification reports show that XGBoost achieves a good balance between precision and recall for both classes. For the train set, precision is around 0.81 for class 0 and 0.80 for class 1, and recall is 0.76 and 0.85 respectively. On the test set, precision and recall remain close to these values, which indicates that the model generalizes well without overfitting heavily. This balance is crucial when both false positives and false negatives matter.

#### **High Accuracy with Generalization**

The train accuracy is approximately 0.805, and the test accuracy is about 0.793. The difference between train and test accuracy is small, suggesting that the model captures the patterns in the data effectively without overfitting. Many simpler models like logistic regression or decision trees can either underfit or overfit in such cases, leading to lower generalization performance.

#### **Ability to Handle Imbalanced Data**

Your dataset seems moderately imbalanced (the support shows 100k+ for class 0 and 117k+ for class 1 in training). XGBoost uses gradient boosting with decision trees, which can naturally handle class imbalance better than single models because each new tree focuses on the errors of the previous ones.

#### **Robustness to Feature Interactions and Non-Linearity**

XGBoost can capture complex patterns because it builds an ensemble of trees, each learning from the residual errors of the previous trees. If your features have non-linear relationships or interactions that simpler models cannot easily capture, XGBoost excels. That's likely why it performs better than linear models or shallow trees.

#### **Regularization and Parameter Tuning**

XGBoost includes built-in regularization parameters like `gamma`, `max_depth`, and `learning_rate`, which help prevent overfitting and optimize the model performance. Proper tuning of these parameters can significantly improve both precision and recall, which seems evident in your results. The confusion matrices show that the number of misclassified samples is reasonably low compared to the total, indicating that the model learned the underlying structure effectively.

#### **Ensemble Effect**

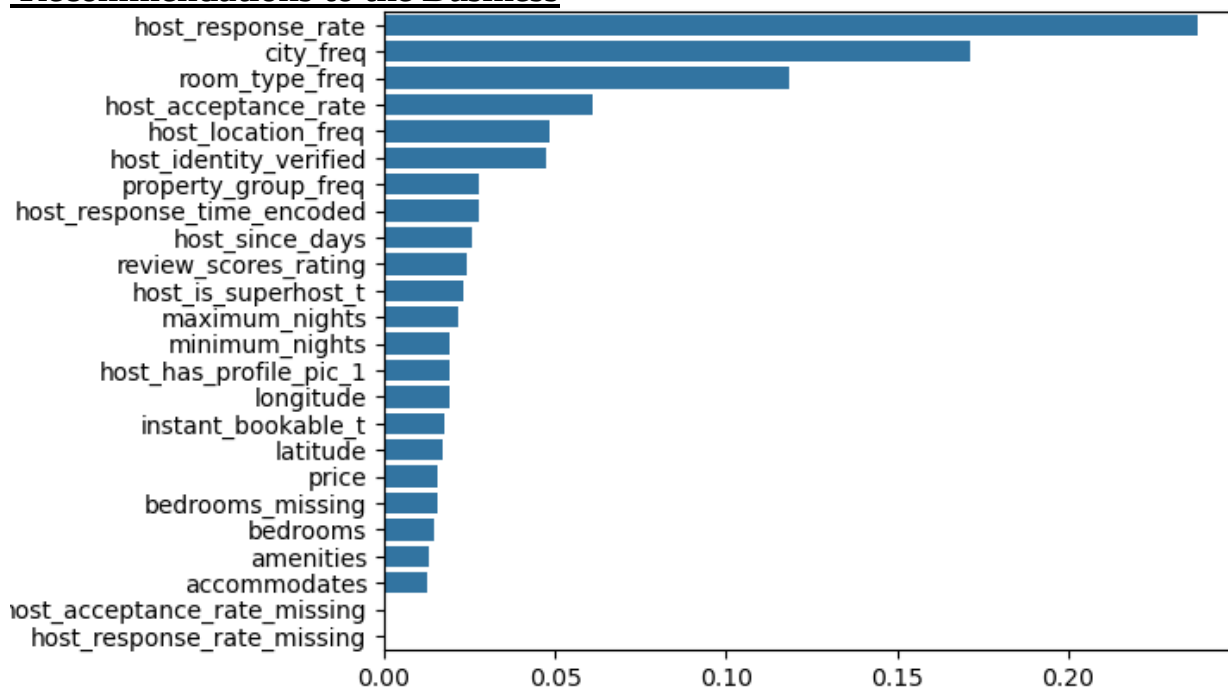
Being a boosting method, XGBoost combines multiple weak learners (trees) to create a strong learner. Each tree corrects the mistakes of the previous ones, which results in improved performance over standalone models like a single decision tree or logistic regression. This explains why it outperforms simpler classifiers in both accuracy and F1 score.

## Final Model:

| Metric    | Class 0 (Old) | Class 1 (New) | Accuracy |
|-----------|---------------|---------------|----------|
| Precision | 0.81          | 0.80          | 0.805    |
| Recall    | 0.76          | 0.85          | 0.805    |
| F1-Score  | 0.78          | 0.82          | 0.805    |

| Metric    | Class 0(Old) | Class 1 (New) | Accuracy |
|-----------|--------------|---------------|----------|
| Precision | 0.79         | 0.80          | 0.794    |
| Recall    | 0.75         | 0.83          | 0.794    |
| F1-Score  | 0.77         | 0.81          | 0.794    |

## Recommendations to the Business



### 1. **Improve host responsiveness (Top priority)**

Host response rate is the most important feature, meaning faster and more consistent replies strongly increase the likelihood of better performance.

# greatlearning

Recommendation: Encourage hosts to enable app notifications, use auto-responses, and respond within minutes to guest inquiries.

## 2. **Focus on high-demand cities and locations**

City frequency and host location frequency rank very high, showing that demand varies significantly by location.

Recommendation: Target marketing and onboarding efforts in high-demand cities and promote listings in popular neighborhoods.

## 3. **Optimize room type strategy**

Room type frequency is a major driver of outcomes, suggesting guests strongly prefer certain room types.

Recommendation: Advise hosts to list high-demand room types (e.g., entire home/private room) where feasible and price them competitively.

## 4. **Increase host acceptance rate and trust signals**

Features like host acceptance rate, identity verification, profile picture, and superhost status contribute meaningfully.

Recommendation: Educate hosts on accepting more bookings, completing identity verification, maintaining profiles, and meeting Superhost criteria.

## 5. **Reduce response time and improve hosting experience**

*Host response time* and *host since days* indicate that experience and quick communication matter.

Recommendation: Provide onboarding support for new hosts and tools that help them respond quickly and manage bookings efficiently.

## 6. **Leverage reviews and ratings**

Review scores rating still plays a role, reinforcing the importance of guest satisfaction.

Recommendation: Encourage hosts to deliver consistent quality and politely request reviews after stays.

## 7. **Optimize stay length and pricing strategy**

Minimum nights, maximum nights, and price influence outcomes but less than host behavior.

Recommendation: Suggest flexible minimum stay rules and competitive pricing, especially in high-demand locations.

So, **improving high-importance, controllable features increases the probability** that a new host will behave like an experienced one.

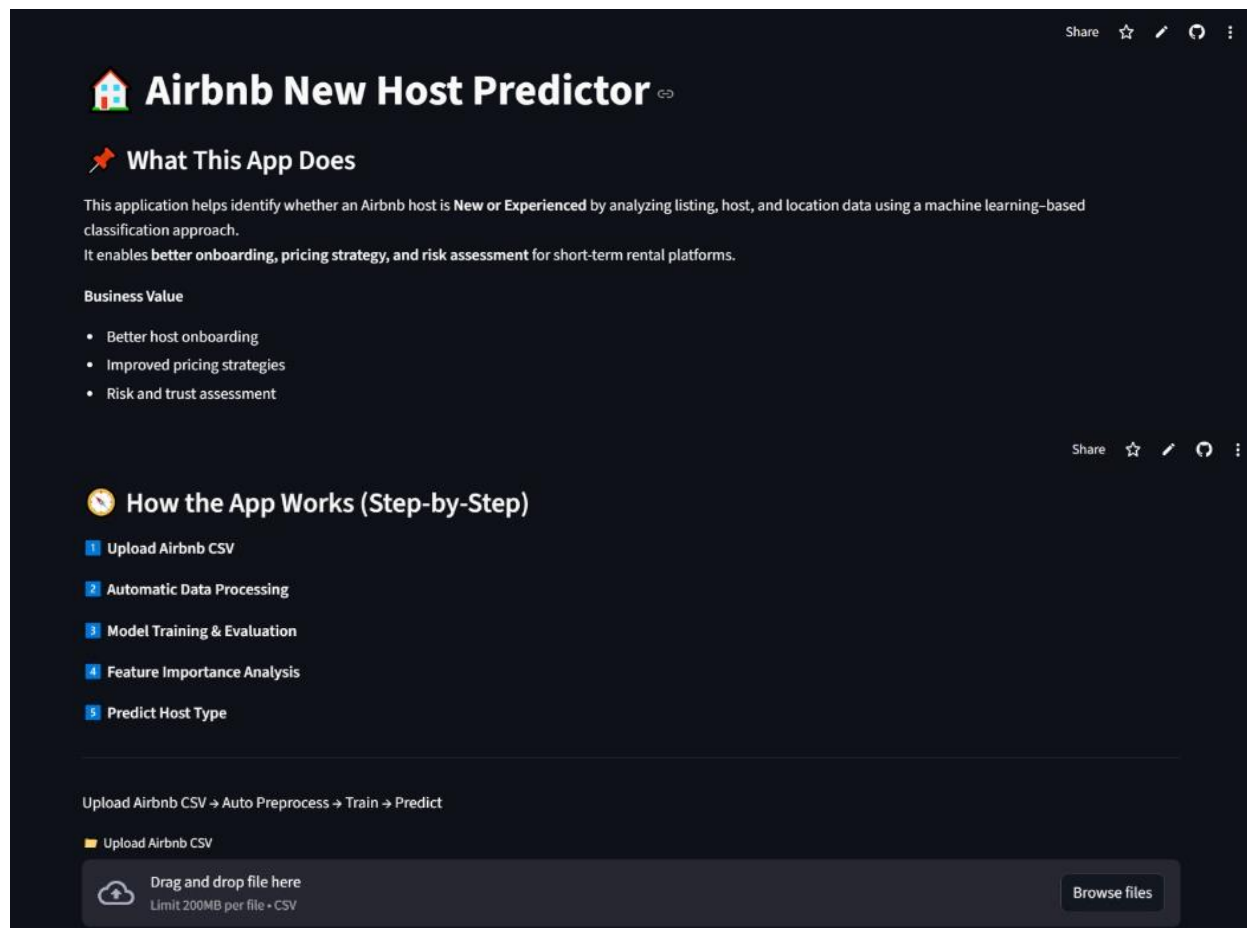
### **What new hosts should focus on (actionable)**

These are **behavioral and controllable** — perfect for new hosts:

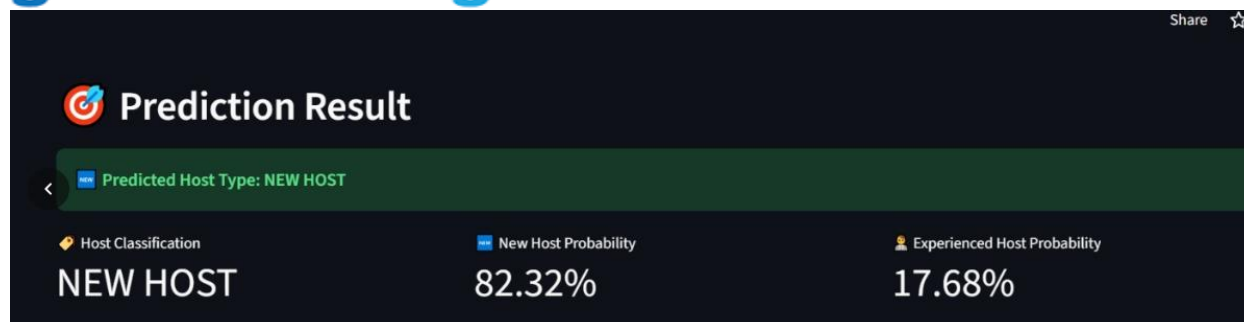
- **Host response rate & response time** → reply fast and consistently
- **Acceptance rate** → avoid unnecessary rejections

## HOST EXPERIENCE PREDICTION SYSTEM(USING STREAMLIT)

This Streamlit-based web application is built to automatically classify Airbnb hosts as New or Experienced using machine learning. The app allows users to upload Airbnb datasets, which are then cleaned, preprocessed, and encoded within the application. A classification model is trained and evaluated in real time, with feature importance displayed to explain key factors influencing predictions. An interactive interface enables users to input listing, host, and location details to instantly predict host type, making the entire workflow end-to-end and user-friendly







The application was developed to eliminate manual analysis and support data-driven business decisions. By identifying host experience levels, it helps businesses improve host onboarding, targeted support for new hosts, optimized pricing strategies, and risk assessment. Streamlit enables rapid deployment, easy accessibility, and quick experimentation, allowing stakeholders to convert raw Airbnb data into actionable insights with minimal technical effort.

## CONCLUSION

XGBoost performed best because it can handle imbalanced data, capture complex non-linear relationships, and generalize well thanks to regularization and boosting. The performance metrics—good balance of precision and recall, small train-test accuracy gap, and relatively low misclassifications—support this. Essentially, it's the combination of boosting, regularization, and flexibility that made it outperform simpler classifiers.

## FUTURE SCOPE

### 1. Smarter Host Onboarding & Support

In the future, this model can be integrated directly into the hosting platform to **identify new hosts early** and provide them with personalized onboarding, training, and incentives. New hosts could receive guidance on pricing, listing optimization, and response behavior, improving their success rate and retention.

### 2. Performance & Revenue Forecasting

The prediction system can be extended beyond just “new vs experienced” to **forecast host performance**, such as expected bookings, revenue, or occupancy rates. Platforms can use this to prioritize high-potential hosts and maximize overall platform revenue.

### 3. Dynamic Pricing & Recommendation Engines

By combining host experience predictions with pricing, location, and seasonality data, the system can support **dynamic pricing strategies**. Experienced hosts might be recommended advanced pricing tools, while new hosts can be guided with conservative or promotional pricing models

-----END OF REPORT-----